Goh_AutomatedVehicleControl(1).pdf
PDF

Read Goh's the

已思考 6s

I see the uploaded dissertation: **Jonathan Yan Ming Goh, *Automated Vehicle Control Beyond the Stability Limits*, Stanford PhD dissertation, June 2019**. Goh_AutomatedVehicleControl

Your message seems unfinished: "Read Goh's the..."

Please tell me what you want me to focus on, for example Chapter 2 controller design, Chapter 3 highly dynamic drifting, Chapter 4 safe operating envelope, or how Goh's methods relate to your vehicle simulation / MPC / NDI work.

資料來源

Read Goh's PhD thesis from beginning to the end very carefully without missing any content. Then explain to me in the most detailed way and most rigorous way about Chapter 2 and Chapter 3

已思考 1m 57s

I read Goh's thesis as one complete argument: Chapter 1 motivates why automated vehicles should not be limited to open-loop-stable handling; Chapter 2 builds a controller for sustained drifting along a general path; Chapter 3 extends that controller to very dynamic drift transitions; Chapter 4 then turns the Chapter 3 "loss of authority" insight into a safe operating envelope; Chapter 5 summarizes the expansion of the vehicle's usable operating domain. The thesis is Jonathan Goh's 2019 Stanford dissertation, ***Automated Vehicle Control Beyond the Stability Limits***. Goh_AutomatedVehicleControl

Below I focus deeply on **Chapter 2** and **Chapter 3**.

1. The central idea connecting Chapter 2 and Chapter 3

Goh's thesis is not just "how to drift a car." It is about how to control a vehicle **after the usual stable-handling assumptions break down**.

In normal driving, the controller structure is usually:

$$\text{steering} \rightarrow \text{path tracking}, \qquad \text{throttle/brake} \rightarrow \text{speed tracking}$$

and the vehicle's sideslip is assumed to remain naturally small and stable.

In drifting, that structure fails. The rear tires are saturated; longitudinal and lateral tire forces are strongly coupled; sideslip is large; yaw dynamics are open-loop unstable. Goh's key reorganization is:

$$\text{steering} + \text{rear drive force together} \rightarrow \text{path tracking} + \text{sideslip stabilization}$$

and **speed is not explicitly tracked**. Instead, Goh argues that under the imposed control law, speed behaves like a stable zero dynamic because of the coupling between lateral and longitudinal dynamics. This is the fundamental "shuffle of objectives" in Chapter 2. Chapter 3 then asks: what happens when the desired maneuver is so aggressive that even path tracking and sideslip stabilization cannot both be achieved at the same time? Goh's answer is: **prioritize yaw/sideslip recovery, even if path tracking must temporarily degrade**. This is the step from quasi-equilibrium drifting to highly dynamic drifting. Goh_AutomatedVehicleControl

2. Chapter 2: Controller Design for Drifting Along a General Path

2.1 What Chapter 2 is trying to solve

Before Goh, much of the drifting-control literature focused on stabilizing a vehicle around **one drift equilibrium**. That is useful, but it is not enough for a real autonomous vehicle, because an emergency maneuver or race-like maneuver requires following a **general path** whose curvature and required sideslip change continuously.

Chapter 2's goal is therefore:

$$\text{Design a controller that can drift along a general path, not merely stabilize one drift point.}$$

The path is not necessarily a circle. The car must track lateral position while also maintaining a large desired sideslip. The controller must work far beyond the stable handling envelope.

Goh uses a relatively simple **single-track planar vehicle model**, but he modifies the usual assumptions carefully. He does **not** use small-angle sideslip approximations, because sideslip can be around -40°. He also does **not** ignore rear wheelspeed dynamics, because the rear tires are fully saturated and force direction depends strongly on wheel slip. Goh_AutomatedVehicleControl

2.2 Vehicle states and path coordinates

Goh uses the planar dynamic states

$$r, \quad V, \quad \beta$$

where:

$$r = \text{yaw rate}$$

$$V = \text{magnitude of CG velocity}$$

$$\beta = \text{sideslip angle}$$

He also introduces the **course angle** ϕ, which is the direction of the vehicle velocity vector in the inertial frame. This is very important.

The vehicle heading is usually denoted ψ. The velocity direction is ϕ. In drifting, these are very different because the body is pointed away from the travel direction. The sideslip is approximately the difference between body heading and velocity direction, depending on sign convention.

The key kinematic relation is:

$$\dot{\phi} = \dot{\beta} + r$$

This equation is the heart of Goh's Chapter 2 controller. It says:

$$\text{course rotation rate} = \text{sideslip change rate} + \text{body yaw rate}$$

So if we control $\dot{\phi}$, we control how the velocity vector turns. If we control r relative to $\dot{\phi}$, we control how the body rotates relative to the velocity vector, i.e. the sideslip.

This is why Goh separates:

$$\dot{\phi} \quad \text{for path tracking}$$

and

$$\dot{r} \quad \text{for yaw/sideslip stabilization}$$

rather than trying to directly control steering for path and throttle for speed.

2.3 Curvilinear path tracking

The vehicle is represented relative to a reference path using:

$$s = \text{distance along the reference path}$$

$$e = \text{lateral error from the path}$$

$$\phi_{\text{ref}}(s) = \text{reference path tangent angle}$$

$$\Delta\phi = \phi - \phi_{\text{ref}}$$

The lateral error dynamics are:

$$\dot{e} = V \sin(\Delta\phi)$$

For small course error, Goh uses:

$$\dot{e} \approx V \Delta\phi$$

and

$$\ddot{e} \approx V \Delta\dot{\phi}$$

This approximation is not saying sideslip is small. It is saying the **course error** is small. That is a much weaker and more reasonable assumption during closed-loop path tracking. The vehicle can be at -40° sideslip, but if the velocity vector is tangent to the path, then $\Delta\phi$ is still small.

The path curvature is:

$$\kappa_{\text{ref}} = \frac{d\phi_{\text{ref}}}{ds}$$

and the course-error rate is:

$$\Delta\dot{\phi} = \dot{\phi} - \kappa_{\text{ref}} \begin{pmatrix} V \cos\Delta\phi \\ 1 - \kappa_{\text{ref}} e \end{pmatrix}$$

This is the standard curvilinear-coordinate correction. The denominator $1 - \kappa_{\text{ref}} e$ accounts for the fact that if the vehicle is laterally offset from a curved path, its local path-distance rate differs from the reference centerline distance rate.

2.4 The single-track vehicle dynamics

The force-based single-track model has front lateral force F_{yf}, front longitudinal force F_{xf}, rear lateral force F_{yr}, and rear longitudinal force F_{xr}. For MARTY, a rear-wheel-drive vehicle, Goh assumes:

$$F_{xf} = 0$$

The equations are essentially:

$$\dot{r} = \frac{aF_{yf}\cos\delta + aF_{xf}\sin\delta - bF_{yr}}{I_z}$$

$$\dot{\beta} = \frac{F_{yf}\cos(\delta-\beta) + F_{xf}\sin(\delta-\beta) + F_{yr}\cos\beta - F_{xr}\sin\beta}{mV} - r$$

$$\dot{V} = \frac{-F_{yf}\sin(\delta-\beta) + F_{xf}\cos(\delta-\beta) + F_{yr}\sin\beta + F_{xr}\cos\beta}{m}$$

These equations are important because they show directly why drifting is strongly coupled. At large β, rear longitudinal force F_{xr} affects not only speed but also sideslip/yaw behavior, and rear lateral force F_{yr} affects not only lateral motion but also speed. This is not a normal-driving decoupled system.

The front tire is modeled with a Fiala brush tire model. The rear tire is assumed to be fully saturated, so the rear tire force lies on the friction circle:

$$F_{xr}^2 + F_{yr}^2 = (\mu F_{zr})^2$$

It is useful to parameterize the rear force vector by a thrust angle γ, so conceptually:

$$F_{xr} = \mu F_{zr} \cos\gamma$$

$$F_{yr} = \mu F_{zr} \sin\gamma$$

or equivalently:

$$\tan\gamma = \frac{F_{yr}}{F_{xr}}$$

This makes the rear force direction a control-relevant quantity. In drifting, the rear tire is already saturated, so the drive torque does not simply "add more total force"; it rotates the available force vector around the friction circle.

3. Chapter 2 controller: imposed error dynamics

Goh's controller has a cascade:

$$(e, \Delta\phi, \beta, r) \to (\dot{\phi}_{des}, \dot{r}_{des}) \to (\delta, \gamma_{des}) \to \omega_{des} \to \tau$$

The first stage is model-independent: impose desired error dynamics. The second stage uses model inversion. The third stage maps desired rear force direction to wheelspeed. The final stage uses torque to track wheelspeed. This architecture is summarized in Goh's controller block diagram.

Goh_AutomatedVehicleControl

3.1 Path tracking through course-rate control

Goh imposes stable second-order lateral-error dynamics:

$$\ddot{e}_{des} = -k_p e - k_d \dot{e}$$

Using:

$$\dot{e} \approx V\Delta\phi$$

$$\ddot{e} \approx V\Delta\dot{\phi}$$

he obtains:

$$V\Delta\dot{\phi}_{des} = -k_p e - k_d V\Delta\phi$$

so:

$$\Delta\dot{\phi}_{des} = -\frac{k_p}{V} e - k_d \Delta\phi$$

Then adding the reference-path course rate gives:

$$\dot{\phi}_{des} = -\frac{k_p}{V} e - k_d \Delta\phi + \dot{\phi}_{ref}$$

This is very elegant. It means:

- If the vehicle is outside the path, change the velocity-vector rotation rate to bring it back.
- If the velocity vector is misaligned with the path tangent, correct the course error.
- The controller is not trying to point the vehicle body along the path; it is trying to point the **velocity vector** along the path.

That is exactly what you want in drifting.

3.2 Sideslip stabilization through synthetic yaw-rate control

Sideslip obeys:

$$\dot{\beta} = \dot{\phi} - r$$

Goh imposes first-order sideslip error dynamics:

$$e_\beta = \beta - \beta_{ref}$$

$$\dot{e}_\beta = -k_\beta e_\beta$$

so:

$$\dot{\beta}_{des} = -k_\beta e_\beta + \dot{\beta}_{ref}$$

Since:

$$\dot{\beta}_{des} = \dot{\phi}_{des} - r_{syn}$$

the required synthetic yaw rate is:

$$r_{syn} = \dot{\phi}_{des} + k_\beta e_\beta - \dot{\beta}_{ref}$$

This r_{syn} is the yaw rate that would produce the desired sideslip dynamics, given the course-rate demand.

Then Goh imposes a yaw-rate tracking dynamic:

$$e_r = r - r_{syn}$$

$$\dot{e}_r = -k_r e_r$$

which gives:

$$\dot{r}_{des} = -k_r(r - r_{syn}) + \dot{r}_{syn}$$

This is the second desired state derivative.

So Chapter 2 produces two derivative commands:

$$\dot{\phi}_{des}$$

for path tracking, and

$$\dot{r}_{des}$$

for sideslip/yaw stabilization.

Those are then passed to nonlinear model inversion.

4. Chapter 2 nonlinear model inversion and the "velocity zero dynamic"

The nonlinear model inversion solves:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \quad \mapsto \quad (\delta, \gamma)$$

or equivalently:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \quad \mapsto \quad (\delta, F_{xr}, F_{yr})$$

subject to the tire model and friction circle.

Here is the critical point:

There are **two physical control inputs**:

$$\delta, \quad \gamma$$

and Goh uses them to satisfy **two derivative objectives**:

$$\dot{\phi}_{des}, \quad \dot{r}_{des}$$

Therefore, there is no remaining input to directly control:

$$\dot{V}$$

So speed is left uncontrolled.

This looks dangerous at first. But Goh argues that speed is stabilized as a zero dynamic. At zero tracking error:

$$e = 0, \quad \Delta\phi = 0, \quad e_\beta = 0, \quad e_r = 0$$

so:

$$\dot{\phi}_{des} = \dot{\phi}_{ref}$$

and because:

$$\dot{\phi}_{ref} = V\kappa_{ref}$$

the demanded course rate grows linearly with speed:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

Now suppose speed is too high. Then the required course rate for the same path curvature is too high. On the appropriate branch of the feasible derivative surface, increasing $\dot{\phi}$ corresponds to decreasing $\dot{V}$, so the car slows down. If speed is too low, the demanded course rate is lower, and the resulting dynamics tend to accelerate the car. Thus, speed self-regulates.

More rigorously, Goh studies the feasible surface of state derivatives:

$$(\dot{\phi}, \dot{r}, \dot{V})$$

generated by admissible inputs. The important local condition is:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

along contours of constant $\dot{r}$, with a zero crossing of $\dot{V}$. Goh defines a separatrix on the feasible surface and selects the branch where this stable relationship holds. The thesis reports that this condition holds over a broad range of drifting states, except at very high yaw rates and very low speeds.

Goh_AutomatedVehicleControl

This is one of the most important conceptual results in Chapter 2:

$$\text{In drifting, velocity can be untracked but stable, if path and sideslip are controlled correctly.}$$

This is the opposite of conventional driving control.

5. Chapter 2 projection into feasible space

Sometimes the desired pair:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

is outside the feasible set produced by the actuator and tire limits.

In Chapter 2, Goh handles this in a relatively simple way:

1. Saturate $\dot{\phi}_{des}$ to feasible values.
2. Compute the corresponding $\dot{r}_{des}$.
3. Project the pair into the feasible tangent-space projection along a line of constant $\dot{r}$.

This is described as a naive method, but it roughly prioritizes yaw/sideslip dynamics over lateral path tracking. Chapter 3 later shows that this naive method is not sufficient for highly dynamic transitions.

6. Chapter 2 wheelspeed dynamics: why force is not immediate

A very important part of Chapter 2 is Goh's argument that rear wheelspeed dynamics cannot be ignored during sustained drifting.

A simple force-input controller might say:

$$\tau = RF_{xr}$$

and treat rear longitudinal force as directly commanded. Goh argues this is not accurate enough during drifting.

Because the rear tire is fully sliding, the tire force direction opposes the slip velocity vector. The slip velocity depends on the wheel surface speed $R\omega$. Therefore, changing wheel speed changes the direction of the rear force vector.

Goh derives a relationship of the form:

$$\gamma = \arctan\begin{pmatrix} -(V\sin\beta - br) \\ -V\cos\beta + R\omega \end{pmatrix}$$

and wheel dynamics:

$$I_\omega \dot{\omega} = \tau - RF_{xr}$$

Thus, torque does not instantaneously create the desired rear force. Torque first changes wheel speed; wheel speed changes slip velocity; slip velocity changes force direction; force direction changes vehicle dynamics.

That is why Goh uses a cascaded wheelspeed controller. The high-level controller computes a desired rear thrust angle or rear force. This is mapped to a desired wheelspeed:

$$\gamma_{des} \rightarrow \omega_{des}$$

Then torque is used to track ω_{des}.

The low-level torque command is:

$$\tau_{command} = -k_\omega I_\omega(\omega - \omega_{des,f}) + I_\omega \dot{\omega}_{des,f} + RF_{xr,des}$$

with a first-order filter on ω_{des}:

$$\dot{\omega}_{des,f} = -\frac{1}{t_\omega}(\omega_{des,f} - \omega_{des})$$

This gives a force-generation bandwidth that is fast and predictable enough for the high-level force-based model inversion to remain valid.

Goh gives two reasons this is better than direct force/torque mapping:

First, controlling rear thrust angle γ is less sensitive to errors in μF_z than commanding fixed F_{xr}. If friction or normal load changes, holding F_{xr} can produce a large unwanted change in F_{yr}, whereas holding the force direction changes the force vector more proportionally.

Second, without wheelspeed control, force generation has a large delay. Goh compares rear sliding-tire force rise time with a front tire relaxation-length estimate and finds that the rear force rise time near the drift equilibrium can be more than five times the front lateral force rise time. This is large enough to destabilize or degrade the coordinated steering/throttle controller. Goh_AutomatedVehicleControl

7. Chapter 2 experiments and what they prove

Goh validates the controller on MARTY, a modified 1981 DMC DeLorean with computer-controlled steering and independently driven rear wheels. The test uses a slowly changing drifting path with:

$$\kappa_{ref} \in [1/20,\ 1/7]\ \mathrm{m}^{-1}$$

speed roughly:

$$25 \text{ to } 45\ \mathrm{km/h}$$

and sideslip up to:

$$-40^\circ$$

The reference path is generated as a sequence of drifting equilibrium points. The controller is activated after a conventional entry and then deactivated near the end.

The result with wheelspeed control is strong:

$$\text{maximum lateral deviation} \approx -0.36\ \mathrm{m}$$

$$\text{RMS lateral error} \approx 0.18\ \mathrm{m}$$

$$\text{maximum sideslip deviation} \approx -6.1^\circ$$

$$\text{RMS sideslip error} \approx 2.4^\circ$$

The measured yaw rate tracks the synthetic yaw-rate command well except during the initial drift transition, and the vehicle speed remains close to the quasi-equilibrium value, supporting the stable velocity-zero-dynamics argument. Goh_AutomatedVehicleControl

Then Goh disables wheelspeed control and commands torque more directly. The car still roughly drifts, but the response becomes much more oscillatory. Lateral error, course error, sideslip, yaw rate, steering command, and rear thrust command all show larger oscillations. Goh also reconstructs tire forces from measured vehicle motion and shows Lissajous-style loops between commanded and measured forces, indicating phase lag. With wheelspeed control, the measured-vs-commanded force relationship stays much closer to the ideal one-to-one line. Goh_AutomatedVehicleControl

So Chapter 2 proves three things:

> A general drifting path can be tracked by controlling velocity-vector rotation and yaw acceleration.

> Speed need not be directly controlled; it can be a stable zero dynamic.

> Rear wheelspeed dynamics matter and should be closed-loop controlled.

8. Chapter 3: Execution of Highly Dynamic Drifting Trajectories

Chapter 2 proves that the controller works for slowly changing, quasi-equilibrium drifting paths. Chapter 3 asks a much harder question:

> Can the vehicle execute rapid drift transitions that push the limits of achievable state derivatives?

The benchmark is a fully autonomous **Figure 8 drift**. This is much harder than a steady drift circle because the car must transition from one large-sideslip equilibrium to the opposite large-sideslip equilibrium. That means:

$$\beta < 0 \quad \rightarrow \quad \beta > 0$$

or vice versa, while maintaining high yaw rate and high speed.

Goh states that the Figure 8 requires high yaw rate, high yaw acceleration, large steering slew, and operation near the boundary of feasible state derivatives. Chapter 3 therefore combines trajectory optimization, tangent-space analysis, and a modified controller that prioritizes yaw/sideslip when objectives conflict. Goh_AutomatedVehicleControl

9. Chapter 3 trajectory planning

9.1 What is being planned?

The Figure 8 consists of:

1. A steady-state left drift.
2. A rapid dynamic transition.
3. A steady-state right drift.
4. Another rapid transition back.

The circular drift segments can be treated as equilibrium points. The hard part is the transition between them.

Goh formulates the transition as a nonlinear optimization problem.

The extended state is:

$$X = \begin{bmatrix} r & V & \beta & E & N & \phi & \delta & F_{xr} \end{bmatrix}^T$$

where E, N are global position coordinates, ϕ is course angle, and δ, F_{xr} are included as states so their rates can be constrained.

The input is:

$$U = \begin{bmatrix} \dot{\delta} & \dot{F}_{xr} \end{bmatrix}^T$$

This is important. Goh plans with actuator slew rates, not just actuator values, because the Figure 8 is limited by how quickly steering and rear force can change.

The problem is spatially discretized. Instead of uniform time steps, it uses uniform path-distance steps:

$$ds$$

and the dynamics are divided by:

$$V = \frac{ds}{dt}$$

then integrated using RK4. This is similar in spirit to many racing optimal-control formulations, where distance along the path is the independent variable.

9.2 Endpoint constraints

The start and end of the transition are constrained to be drifting equilibria:

$$\dot{r} = 0, \quad \dot{V} = 0, \quad \dot{\beta} = 0$$

for chosen:

$$\beta_I, \quad \beta_F, \quad \kappa_I, \quad \kappa_F$$

The particular experimental transition uses:

$$\beta_I = -\beta_F = -0.7\ \mathrm{rad}$$

$$\kappa_I = -\kappa_F = \frac{1}{18}\ \mathrm{m}^{-1}$$

and a center-of-rotation spacing:

$$D = 43\ \mathrm{m}$$

The final pose is chosen by using a nominal clothoid-like path as a geometric guide, but the optimized trajectory is constrained only at the beginning and end, not forced to follow the clothoid in between.

9.3 Actuator, torque, and power constraints

Goh constrains:

$$\dot{\delta}$$

$$\dot{F}_{xr}$$

$$\delta$$

$$F_{xr}$$

and also drivetrain torque and power.

The rear tire is assumed saturated, so:

$$F_{xr} = \mu F_{zr} \cos\gamma$$

which gives:

$$\gamma = -\operatorname{sgn}(\alpha_r) \arccos\left(\begin{matrix} F_{xr} \\ \mu F_{zr} \end{matrix}\right)$$

Then the wheel speed can be estimated from the saturated slip geometry:

$$\omega = \frac{1}{R}\left(\begin{matrix} -(V\sin\beta - br) \\ \tan\gamma \end{matrix} + V\cos\beta\right)$$

Torque is approximated over a spatial interval as:

$$\tau_k \approx I_w \frac{\omega_{k+1} - \omega_k}{s_{k+1} - s_k} V_k + R F_{xr,k}$$

and power is:

$$P_k = \tau_k \omega_k$$

The optimization includes constraints such as:

$$0 \le \tau_k \le \tau_{max}$$

$$0 \le P_k \le P_{max}$$

The table in Chapter 3 gives values such as:

$$P_{max} = 250\ \text{kW}$$

$$\tau_{max} = 4.4\ \text{kNm}$$

$$|\dot{\delta}|_{max} = 2\ \text{rad/s}$$

$$|\dot{F}_{xr}|_{max} = 20\ \text{kN/s}$$

$$|\delta|_{max} = 0.66\ \text{rad}$$

The important point is that Goh's plan is not just geometrically possible; it is intended to be physically executable by MARTY's actuators.

9.4 Cost function

The cost function is:

$$J = \sum_{k=1}^{N-1} U_k^T R U_k + \sum_{k=1}^{N} Q\left(|\beta_k - \bar{\beta}| - \Delta\beta^*\right)^2$$

where:

$$\bar{\beta} = \frac{\beta_I + \beta_F}{2}$$

$$\Delta\beta^* = |\beta_I - \bar{\beta}| = |\beta_F - \bar{\beta}|$$

The first term penalizes actuator slew rate, so it encourages smooth steering and rear-force changes.

The second term encourages the sideslip to be near either endpoint sideslip, rather than spending a long time in the middle. A perfect step change in sideslip would have zero cost for that second term. Of course, a true step is impossible due to dynamics and actuator limits, but the cost pushes the optimizer toward a rapid transition.

So Q tunes aggressiveness:

- low Q: smoother, slower sideslip transition;
- high Q: faster sideslip transition, higher yaw rate, faster steering sweep, more actuator stress.

Goh chooses an intermediate value for experiment to balance dynamic aggressiveness with actuator headroom.

10. Chapter 3 tangent-space analysis

This is the most important theoretical part of Chapter 3.

At each vehicle state:

$$x = (r, V, \beta)$$

the constrained actuator set maps through the dynamics to a set of possible derivatives:

$$(\dot{r}, \dot{\phi}, \dot{V})$$

Goh calls this set the **tangent space**. Because there are two main control inputs, the feasible derivative set is a two-dimensional surface embedded in three-dimensional derivative space.

The controller from Chapter 2 commands:

$$\dot{\phi}_{des}$$

and

$$\dot{r}_{des}$$

so it is useful to project the tangent space onto the:

$$\dot{\phi} - \dot{r}$$

plane.

The boundary of that projection tells you:

$$\text{which combinations of path-course rotation and yaw acceleration are physically achievable.}$$

Chapter 3 shows that the Figure 8 trajectory must operate close to this boundary. Goh argues that you cannot simply plan with a large safety margin everywhere, because one of the tangent-space boundaries is caused by a fold in the surface. If you leave a large buffer from that fold in the $\dot{\phi} - \dot{r}$ projection, you may eliminate the ability to access negative $\dot{V}$. That would force the car to only accelerate or maintain speed in certain regions, which is too restrictive for dynamic drifting.

This is a deep point:

$$\text{Highly dynamic drifting is not merely poor planning with insufficient margin. It fundamentally requires operating near derivative limits.}$$

11. Why yaw/sideslip must be prioritized

Goh then studies how the tangent-space boundary changes with sideslip.

As sideslip magnitude grows, especially near the counter-steering limit, the feasible tangent space shrinks. The reason is front tire and steering saturation.

The maximum accessible front slip angle is approximately:

$$\alpha_{F,max} = \arctan\left(\begin{matrix} V\sin\beta + ar \\ V\cos\beta \end{matrix}\right) + \delta_{max}$$

When sideslip becomes too large, even maximum counter-steering cannot produce the desired front tire force range. The vehicle loses yaw authority.

In Goh's example, as $|\beta|$ grows from about 40° toward 50°, the feasible set shrinks dramatically. At very high sideslip, the vehicle may have no input combination capable of decreasing yaw rate in the needed direction. That is the beginning of a spin.

This is the core physical insight of Chapter 3:

$$\text{The dangerous state is not simply "outside open-loop stability."}$$

The dangerous state is:

$$\text{large sideslip where the controller loses authority to reduce sideslip/yaw rate.}$$

This insight later becomes the foundation of Chapter 4's recovery envelope.

12. Chapter 3 yaw-prioritizing projection

When the nominal controller demand:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

falls outside the feasible tangent-space boundary, the vehicle cannot simultaneously satisfy:

1. path tracking, and
2. sideslip/yaw stabilization.

Goh argues that sideslip/yaw must be prioritized, because path error is recoverable later, but loss of yaw authority can cause a spin.

He writes this as an optimization problem:

$$\min\ \dot{\phi}(x,u) - \dot{\phi}_{des}$$

subject to:

$$\dot{r}(x,u) = \dot{r}_{des}(x, p, \dot{\phi}_{des})$$

$$u_{min} \le u \le u_{max}$$

This says:

$$\text{keep yaw/sideslip dynamics correct, and sacrifice course-rate tracking as little as necessary.}$$

However, solving this nonlinear constrained optimization online would be expensive and unreliable. Goh therefore transforms it into a geometric projection problem.

The key is that the yaw-control law imposes an affine relation between $\dot{r}_{des}$ and $\dot{\phi}_{des}$:

$$\dot{r}_{des} = \left[k_r(-r + k_\beta e_\beta - \dot{\beta}_{ref}) + \dot{r}_{syn}\right] + k_r \dot{\phi}_{des}$$

So in the:

$$\dot{\phi} - \dot{r}$$

plane, all points that preserve the yaw-control law lie on a line with slope k_r.

Therefore:

- If the nominal command is feasible, use it.
- If it is infeasible, project it onto the tangent-space boundary **along the yaw-control-law line**.
- This preserves the coupling between path-course command and yaw command.
- If the line does not intersect the feasible set, hold the previous actuator solution, corresponding to a maximal recovery action.

This is the **yaw-stability-preserving projection**. Goh contrasts it with a naive projection, which simply projects along the $\dot{\phi}$ axis while keeping the original $\dot{r}$. That naive projection looks as if it preserves yaw acceleration, but it actually ignores that $\dot{r}_{des}$ itself depends on $\dot{\phi}_{des}$. Therefore it does not truly preserve the intended yaw/sideslip dynamics. Goh_AutomatedVehicleControl

This is probably the most important control detail in Chapter 3.

13. Tangent-space boundary computation

To implement the yaw-preserving projection, Goh needs the boundary of the feasible tangent-space projection.

For a left-hand drift case, he divides the outer boundary into four segments.

The top point corresponds to maximum yaw acceleration:

$$\dot{r}_{max}$$

which occurs when the front yaw contribution is maximized and rear lateral force is minimized. Goh defines:

$$\delta^{+} = \arg\max_{\delta}(F_{yf}\cos\delta)$$

and at the top point:

$$F_{yr} = 0, \quad F_{xr} = \mu F_{zr}$$

Another boundary is the fold/separatrix already discussed in Chapter 2. Goh derives an explicit relation between steering and rear thrust angle:

$$\gamma = \arctan\begin{pmatrix} q\cos\beta + mVb/I_z \\ -q\sin\beta \end{pmatrix}$$

where q comes from the local derivative relationship between $\dot{r}$, $\dot{\phi}$, and steering. This boundary is only valid where:

$$\frac{\partial \dot{r}}{\partial \delta} > 0$$

and

$$\frac{\partial \phi}{\partial \delta} > 0$$

The rest of the boundary is formed by constant-input contours, such as fixed steering or fixed rear force at saturation.

The rigorous meaning is: Goh does not sample the input space blindly. He uses the geometry of the model to compute the boundary efficiently enough for real-time projection.

14. Chapter 3 experiments

Goh runs two main experiments:

1. **Yaw-prioritization active:** vehicle completes two laps of the Figure 8.
2. **Yaw-prioritization disabled / naive projection:** vehicle spins out.

The experiment also uses online parameter estimation for rear tire friction and front cornering stiffness, because real-world friction and tire parameters are hard to know exactly.

With yaw prioritization active:

$$\text{yaw rate peaks around } 2 \text{ rad/s}$$

$$\text{steering sweeps about } 75^{\circ} \text{ in less than } 1 \text{ s}$$

$$\text{speed reaches about } 50 \text{ km/h}$$

The path tracking is not perfect during the transitions. Lateral error peaks around:

$$1.3 \text{ m}$$

near the end of transitions, but then reduces to generally less than:

$$0.4 \text{ m}$$

once the vehicle recovers enough control authority. Sideslip overshoot is around:

$$5^{\circ}$$

at the end of transitions.

This is exactly the intended trade-off: allow temporary path error to preserve yaw/sideslip authority.

With naive projection, the vehicle initially follows similarly, but after the peak yaw-rate region the states diverge. Sideslip overshoots, the tangent space shrinks, yaw authority is lost, and the vehicle spins out. Goh's comparison shows that the difference is not merely tracking accuracy; it is the difference between remaining inside a recoverable control-authority region and leaving it. Goh_AutomatedVehicleControl Goh_AutomatedVehicleControl

15. The deepest conceptual difference between Chapter 2 and Chapter 3

Chapter 2 says:

$$\text{Use } \dot{\phi} \text{ for path tracking and } \dot{r} \text{ for sideslip stabilization.}$$

Chapter 3 says:

$$\text{When } \dot{\phi} \text{ and } \dot{r} \text{ cannot both be achieved, protect } \dot{r}\text{/sideslip authority first.}$$

That is the progression.

Chapter 2 is about **control structure**.

Chapter 3 is about **control authority management**.

Chapter 2 shows drifting can be controlled along a general path.

Chapter 3 shows dynamic drifting can only be controlled if the controller understands the feasible derivative set and makes the correct trade-off near its boundary.

16. Why this matters for your own vehicle simulation / OCP / MPC work

For your high-fidelity vehicle model, Goh's thesis suggests a very important controller/OCP lesson.

A high-fidelity model alone is not enough. The controller must know which state derivatives are achievable at the current state. In your case, with a 14-DOF model and 3D ribbon road, the analogous object is not just Goh's planar tangent space:

$$(\dot{r}, \dot{\phi}, \dot{V})$$

but a higher-dimensional constrained derivative set involving:

$$\dot{v}_x, \dot{v}_y, \dot{v}_z, \dot{p}, \dot{q}, \dot{r},$$

suspension states, wheel speeds, normal-load constraints, road-frame kinematics, and tire friction limits.

But the principle is the same:

$$\text{For extreme maneuvers, the controller should not only track references; it must manage recoverability.}$$

For your direct optimal control plus MPC tracking workflow, the Goh-style lesson is:

1. The offline OCP may generate aggressive trajectories near physical limits.
2. The online or closed-loop tracker must know when exact tracking is infeasible.
3. In those moments, it should not blindly minimize all tracking errors equally.
4. It should prioritize the states that preserve control authority and avoid unrecoverable motion.
5. For drifting-like behavior, yaw/sideslip recovery is more important than instantaneous path error.

That is why Chapter 3 is so important: it shows that successful extreme vehicle control is not just "better tracking." It is **correct prioritization when tracking objectives conflict**.

資料來源

For Chapter 2.2 and 2.3, I don't understand the control design at all. Please explain to me in the most detailed way about how the author designed the controller, why there is nonlinear inversion, what is the state derivatives, why the stability of V and how that is concerned, zero dynamics, separatrix, projection into feasible space, and how the wheel dynamics are considered. I don't understand at all from not only the top level view but also the very detailed details of each aspect. It seems Chapter 2.2 and 2.3 are extremely important for the whole dissertation, so I want to understand and learn as best as I can, and I have to rely on your explanations.

已思考 1m 21s

Below I will rebuild Chapter 2.2–2.3 from the ground up. The key is that Goh is **not** designing a normal path-tracking controller. He is designing a controller for a car whose rear tires are saturated, whose yaw/sideslip dynamics are open-loop unstable, and whose throttle and steering are strongly coupled. The thesis says the Chapter 2 controller is a cascade: imposed error dynamics → nonlinear model inversion → tire-slip/wheelspeed mapping → wheelspeed control → vehicle dynamics. Goh_AutomatedVehicleControl

1. The whole controller in one sentence

Goh's controller does this:

$$(e, \Delta\phi, \beta, r, V) \implies (\dot{\phi}_{des}, \dot{r}_{des}) \implies (\delta, \gamma_{des}) \implies \omega_{des} \implies \tau$$

Meaning:

$$\text{tracking errors} \to \text{desired motion derivatives} \to \text{steering and rear force direction} \to \text{desired rear wheel speed} \to \text{motor torque}$$

This is very different from normal driving, where we usually think:

$$\delta \to \text{path tracking}, \qquad \tau \to \text{speed control}$$

In drifting, that normal separation is wrong. The rear tire is saturated, so throttle changes not only longitudinal force but also lateral force and yaw moment. Steering also affects speed because the front lateral force is not simply perpendicular to the velocity vector at large sideslip. That is why Goh does **coordinated steering/throttle control**, not separate steering-for-path and throttle-for-speed control.

2. The most important variables

Goh uses a planar single-track model with three main dynamic states:

$$r = \text{yaw rate}$$

$$V = \text{speed magnitude of the CG velocity vector}$$

$$\beta = \text{sideslip angle}$$

Then he introduces path-tracking variables:

$$e = \text{lateral error from the reference path}$$

$$s = \text{distance along the reference path}$$

$$\phi = \text{course angle, i.e. direction of the velocity vector}$$

$$\phi_{ref} = \text{reference path tangent angle}$$

$$\Delta\phi = \phi - \phi_{ref}$$

This distinction is absolutely critical:

$$\psi = \text{vehicle body heading}$$

$$\phi = \text{direction the vehicle is actually moving}$$

In normal driving, ψ and ϕ are close. In drifting, they can be very different. The car body may point strongly sideways, but the velocity vector still follows the path. So Goh tracks the path using ϕ, not body heading ψ. The thesis defines the course angle ϕ as the direction of the CG velocity vector, and its derivative $\dot{\phi}$ is the rotation rate of the velocity vector; it is linked to sideslip and yaw rate by $\dot{\phi} = \dot{\beta} + r$.

Goh_AutomatedVehicleControl

That equation can be rearranged as:

$$\dot{\beta} = \dot{\phi} - r$$

This tiny equation is the core of the controller.

It means:

$$\text{sideslip changes when velocity-vector rotation and body-yaw rotation differ.}$$

If the body yaw rate r equals the velocity-vector rotation rate $\dot{\phi}$, sideslip stays constant. If r is larger than $\dot{\phi}$, the body rotates faster than the velocity vector. If r is smaller than $\dot{\phi}$, the velocity vector rotates faster than the body.

So Goh's insight is:

$$\dot{\phi} \text{ can be used for path tracking, while } r \text{ relative to } \dot{\phi} \text{ can be used for sideslip control.}$$

3. Why the controller first produces "desired state derivatives"

The controller does **not** first ask:

$$\text{What steering angle do I need?}$$

Instead it asks:

$$\text{How should the vehicle motion change right now?}$$

That is why the first output of the high-level controller is:

$$\dot{\phi}_{des}$$

and

$$\dot{r}_{des}$$

These are "state derivatives" or "motion derivatives."

More specifically:

$$\dot{\phi}_{des} = \text{desired course-rate, i.e. how fast the velocity vector should rotate}$$

$$\dot{r}_{des} = \text{desired yaw acceleration, i.e. how fast yaw rate should change}$$

The reason this is powerful is that it separates the **control objective** from the **vehicle actuation problem**.

First, Goh says:

$$\text{For good path tracking and sideslip control, the vehicle should have these desired derivatives.}$$

Then he says:

$$\text{Now invert the nonlinear vehicle model to find the steering and rear force direction that produce those derivatives.}$$

This is why nonlinear inversion appears later.

4. Path tracking: why $\dot{\phi}$ is the path-control variable

The lateral error dynamics are:

$$\dot{e} = V \sin(\Delta\phi)$$

For small course error,

$$\dot{e} \approx V\Delta\phi$$

and differentiating gives approximately:

$$\ddot{e} \approx V\Delta\dot{\phi}$$

Goh is **not** assuming small sideslip here. He is assuming small **course error** $\Delta\phi$. That is reasonable in closed-loop path tracking: the car can be at -40° sideslip while its velocity vector is still nearly tangent to the path. The thesis explicitly uses the approximations $\dot{e} \approx V\Delta\phi$ and $\ddot{e} \approx V\Delta\dot{\phi}$, with the assumption justified because $\Delta\phi$ is driven small in closed-loop operation. Goh_AutomatedVehicleControl

Goh imposes stable second-order lateral-error dynamics:

$$\ddot{e}_{des} = -k_p e - k_d \dot{e}$$

Using:

$$\dot{e} \approx V\Delta\phi$$

and

$$\ddot{e} \approx V\Delta\dot{\phi}$$

he obtains:

$$V\Delta\dot{\phi}_{des} = -k_p e - k_d V\Delta\phi$$

Divide by V:

$$\Delta\dot{\phi}_{des} = -\frac{k_p}{V} e - k_d \Delta\phi$$

But the actual desired course rate must also include the reference path's own rotation rate:

$$\dot{\phi}_{des} = \Delta\dot{\phi}_{des} + \dot{\phi}_{ref}$$

Therefore:

$$\dot{\phi}_{des} = -\frac{k_p}{V} e - k_d \Delta\phi + \dot{\phi}_{ref}$$

This equation says:

If the car is laterally away from the path, rotate the velocity vector back toward the path. If the velocity vector is not aligned with the path tangent, correct that course error. And even with zero error, rotate the velocity vector at the rate required by the reference path curvature.

For a path with curvature κ_{ref}, the reference course-rate is:

$$\dot{\phi}_{ref} = \kappa_{ref} \frac{V\cos\Delta\phi}{1 - \kappa_{ref} e}$$

At zero path error and zero course error:

$$e = 0, \quad \Delta\phi = 0$$

so:

$$\dot{\phi}_{ref} = V\kappa_{ref}$$

This becomes very important for the velocity zero-dynamics argument later.

5. Sideslip stabilization: why yaw rate is a "synthetic input"

Now Goh has already chosen $\dot{\phi}_{des}$ for path tracking. But sideslip must also be controlled.

The kinematic relation is:

$$\dot{\beta} = \dot{\phi} - r$$

So once $\dot{\phi}_{des}$ is chosen, the only way to control $\dot{\beta}$ is to make the body yaw rate r appropriate.

Goh imposes stable first-order sideslip error dynamics:

$$e_\beta = \beta - \beta_{ref}$$

$$\dot{e}_\beta = -k_\beta e_\beta$$

Thus:

$$\dot{\beta}_{des} = -k_\beta e_\beta + \dot{\beta}_{ref}$$

Now use:

$$\dot{\beta}_{des} = \dot{\phi}_{des} - r_{syn}$$

Solve for the yaw rate that would produce the desired sideslip dynamics:

$$r_{syn} = \dot{\phi}_{des} - \dot{\beta}_{des}$$

Substitute $\dot{\beta}_{des}$:

$$r_{syn} = \dot{\phi}_{des} + k_\beta e_\beta - \dot{\beta}_{ref}$$

This r_{syn} is called a synthetic yaw rate. It is not a direct actuator. It is the yaw rate that the car **should have** so that the desired sideslip dynamics happen.

Then Goh closes another first-order loop around yaw-rate error:

$$e_r = r - r_{syn}$$

$$\dot{e}_r = -k_r e_r$$

So:

$$\dot{r} - \dot{r}_{syn} = -k_r(r - r_{syn})$$

Therefore:

$$\dot{r}_{des} = -k_r(r - r_{syn}) + \dot{r}_{syn}$$

The thesis gives these exact steps: impose first-order sideslip dynamics, compute the synthetic yaw rate from $\dot{\beta} = \dot{\phi} - r$, then impose first-order tracking of r_{syn} to produce $\dot{r}_{des}$. Goh_AutomatedVehicleControl

The physical meaning is:

$$\dot{\phi}_{des} \text{ is chosen to fix path error.}$$

$$r_{syn} \text{ is chosen so that sideslip changes correctly.}$$

\boxed{ \dot{r}_{des} \text{ is chosen so the real yaw rate tracks } r_{syn}.} }

This is why Goh says path tracking is handled by rotating the velocity vector, while sideslip is stabilized by yawing the vehicle body faster or slower than the velocity vector.

6. What nonlinear inversion means here

After the imposed-error-dynamics step, Goh has:

$$\dot{\phi}_{des}$$

and

$$\dot{r}_{des}$$

But the real vehicle does not accept $\dot{\phi}$ and $\dot{r}$ as commands. The real vehicle accepts:

$$\delta = \text{steering angle}$$

$$\tau = \text{rear drivetrain torque}$$

At the force-based model level, Goh uses the rear force direction, or thrust angle,

$$\gamma$$

instead of torque directly.

So he needs to solve:

$$(\delta, \gamma) \quad \mapsto \quad (\dot{\phi}, \dot{r})$$

backward:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \quad \mapsto \quad (\delta, \gamma_{des})$$

That backward solution is the nonlinear inversion.

It is nonlinear because the model includes trigonometric geometry, large sideslip, large steering angles, nonlinear front tire behavior, and rear friction-circle saturation. The force-based single-track model is numerically inverted to map $(\dot{\phi}_{des}, \dot{r}_{des})$ into steering angle δ and rear thrust angle γ. Goh_AutomatedVehicleControl

The vehicle equations include:

$$\dot{r} = \frac{aF_{yf}\cos\delta + aF_{xf}\sin\delta - bF_{yr}}{I_z}$$

For MARTY, rear-wheel drive is assumed:

$$F_{xf} = 0$$

The sideslip and speed equations are:

$$\dot{\beta} = \frac{F_{yf}\cos(\delta - \beta) + F_{xf}\sin(\delta - \beta) + F_{yr}\cos\beta - F_{xr}\sin\beta}{mV} - r$$

$$\dot{V} = \frac{-F_{yf}\sin(\delta - \beta) + F_{xf}\cos(\delta - \beta) + F_{yr}\sin\beta + F_{xr}\cos\beta}{m}$$

The course-rate is then:

$$\dot{\phi} = \dot{\beta} + r$$

So for each candidate δ, γ, the model predicts:

$$\dot{r}, \quad \dot{\beta}, \quad \dot{\phi}, \quad \dot{V}$$

The inversion chooses δ, γ so that:

$$\dot{\phi}(\delta, \gamma) = \dot{\phi}_{des}$$

$$\dot{r}(\delta, \gamma) = \dot{r}_{des}$$

The leftover derivative is:

$$\dot{V}(\delta, \gamma)$$

That leftover $\dot{V}$ is not directly chosen.

7. Why speed V is not explicitly controlled

This is one of the most confusing but most important parts.

There are basically three things you might want to control:

$$e = \text{path error}$$

$$\beta = \text{sideslip}$$

$$V = \text{speed}$$

But there are only two main inputs:

$$\delta$$

$$\gamma \text{ or rear torque}$$

So you cannot independently impose arbitrary dynamics on all three:

$$e, \quad \beta, \quad V$$

Goh chooses to spend the two input "degrees of freedom" on:

$$\dot{\phi}_{des} \quad \text{for path tracking}$$

and

$$\dot{r}_{des} \quad \text{for sideslip/yaw stabilization}$$

That means speed is not directly tracked.

This sounds dangerous. What if speed runs away? What if speed decays to zero?

Goh's answer is: in drifting, because lateral and longitudinal dynamics are strongly coupled, speed can become a **stable zero dynamic** under this controller. The thesis explicitly says the speed is not explicitly regulated; instead, high-sideslip coupling stabilizes velocity under the imposed control law. Goh_AutomatedVehicleControl

8. What "zero dynamics" means here

In nonlinear control, suppose your controller is designed to drive some outputs to zero. Here the controlled errors are essentially:

$$e \to 0$$

$$\Delta\phi \to 0$$

$$e_\beta = \beta - \beta_{ref} \to 0$$

$$e_r = r - r_{syn} \to 0$$

The **zero dynamics** are the internal dynamics that remain when the controlled outputs are exactly zero.

Here the important internal variable is speed V.

So Goh asks:

When

$$e = 0, \quad \Delta\phi = 0, \quad e_\beta = 0, \quad e_r = 0$$

what happens to V?

At zero path/course error:

$$\dot{\phi}_{des} = \dot{\phi}_{ref} = V\kappa_{ref}$$

Also, at zero yaw/sideslip error:

$$\dot{r}_{des} = \dot{r}_{ref}$$

So the controller's demanded course rate depends linearly on speed:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

The thesis states this zero-error condition as $\dot{\phi}_{des} = \dot{\phi}_{ref} = V\kappa_{ref}$ and $\dot{r}_{des} = \dot{r}_{ref}$. Goh_AutomatedVehicleControl

Now imagine the car is on a reference drifting path of fixed curvature κ_{ref}.

If V becomes too high, then:

$$\dot{\phi}_{des} = V\kappa_{ref}$$

becomes too high. The controller asks the velocity vector to rotate faster. On the selected branch of the feasible derivative surface, asking for more $\dot{\phi}$ tends to produce **negative** $\dot{V}$. So speed decreases.

If V becomes too low, then $\dot{\phi}_{des}$ becomes too low. On the selected branch, that tends to produce **positive** $\dot{V}$. So speed increases.

That is the stable-speed mechanism.

Very roughly:

$$V \uparrow \Rightarrow \dot{\phi}_{des} \uparrow \Rightarrow \dot{V} < 0 \Rightarrow V \downarrow$$

$$V \downarrow \Rightarrow \dot{\phi}_{des} \downarrow \Rightarrow \dot{V} > 0 \Rightarrow V \uparrow$$

So V is not ignored. It is measured and appears inside the controller. But there is no independent speed reference loop. Speed is stabilized indirectly by the geometry of the coupled drifting dynamics.

9. The condition for stable V

Goh says velocity should be stable if two things hold along a constant-$\dot{r}$ contour:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

and

$$\dot{V} \text{ has a zero crossing}$$

The first condition means:

$$\text{If the demanded course-rate increases, speed acceleration decreases.}$$

The second condition means:

$$\text{There exists some point where } \dot{V} = 0.$$

Together, they mean the speed dynamics have an equilibrium and the feedback sign is restoring.

Think of a simple scalar system:

$$\dot{V} = f(V)$$

A stable equilibrium V^* needs:

$$f(V^*) = 0$$

and locally:

$$\frac{df}{dV} < 0$$

Here the path controller creates:

$$\dot{\phi}_{des} = V \kappa_{ref}$$

so increasing V increases the requested $\dot{\phi}$. Therefore:

$$\frac{d\dot{V}}{dV} = \frac{\partial \dot{V}}{\partial \dot{\phi}} \frac{d\dot{\phi}_{des}}{dV}$$

Since:

$$\frac{d\dot{\phi}_{des}}{dV} = \kappa_{ref} > 0$$

stability requires:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

That is exactly Goh's condition.

This is why $\partial \dot{V} / \partial \dot{\phi} < 0$ matters.

It is not an arbitrary mathematical curiosity. It is the slope that decides whether speed self-corrects or self-diverges.

10. What the feasible derivative surface is

At a fixed vehicle state:

$$(r, V, \beta)$$

you can vary the inputs:

$$\delta, \quad \gamma$$

For every possible input pair, the vehicle model produces a derivative triple:

$$(\dot{\phi}, \dot{r}, \dot{V})$$

So all possible input pairs form a surface in derivative space:

$$(\delta, \gamma) \mapsto (\dot{\phi}, \dot{r}, \dot{V})$$

This is the "surface of possible state derivatives" in Chapter 2.

It is useful to imagine a 3D plot with axes:

$$\dot{\phi}$$

$$\dot{r}$$

$$\dot{V}$$

For a fixed state, not every point in this 3D space is achievable. You only have two inputs, so the achievable set is a 2D surface.

The nonlinear inversion is choosing a point on this surface with the desired coordinates:

$$\dot{\phi} = \dot{\phi}_{des}$$

$$\dot{r} = \dot{r}_{des}$$

Then $\dot{V}$ is whatever height the surface has at that point.

The thesis shows representative feasible derivative surfaces and contour plots of $\dot{V}$, $\dot{r}$, and $\dot{\phi}$ for MARTY; it also says the surface shape determines whether the velocity zero-dynamics are stable.

Goh_AutomatedVehicleControl

11. Why there can be more than one inverse solution

For the same desired pair:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

there may be two possible actuator combinations:

$$(\delta_1, \gamma_1)$$

and

$$(\delta_2, \gamma_2)$$

that give the same $\dot{\phi}$ and $\dot{r}$, but different $\dot{V}$.

This happens because the feasible derivative surface can fold over itself when projected onto the $(\dot{\phi}, \dot{r})$ plane.

That is why Goh discusses a **separatrix**.

12. What the separatrix is

The separatrix is the boundary between two branches of the derivative surface.

Goh defines it using the condition where the mapping from inputs to desired derivatives becomes locally singular. In simplified form, the determinant-like condition is:

$$\frac{\partial \dot{\phi}}{\partial \delta} \frac{\partial \dot{r}}{\partial F_{xr}} - \frac{\partial \dot{\phi}}{\partial F_{xr}} \frac{\partial \dot{r}}{\partial \delta} = 0$$

This is the place where the projection of the input surface onto the $(\dot{\phi}, \dot{r})$ plane folds. The thesis defines the separatrix using this derivative condition and says that constraining inversion to the upper surface gives stable velocity zero-dynamics and makes the mapping one-to-one. Goh_AutomatedVehicleControl

The intuitive meaning:

There are two possible branches:

$$\text{upper branch: } \frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

$$\text{lower branch: } \frac{\partial \dot{V}}{\partial \dot{\phi}} > 0$$

The upper branch gives stable speed behavior.

The lower branch could give unstable speed behavior.

So Goh says: during nonlinear inversion, choose the branch above the separatrix.

That does two things:

1. It gives the desired stable $\dot{V}$ zero dynamics.
2. It avoids ambiguous inverse solutions, because the selected branch makes the mapping one-to-one.

This is very important. The controller is not just solving any inverse. It is solving the inverse on the physically useful branch.

13. Why the separatrix must lie below $\dot{V} = 0$

The stable-speed argument requires that along the selected branch, there is a zero crossing of $\dot{V}$, and the slope around it has the right sign.

Goh's phrase "separatrix lies below the $\dot{V} = 0$ plane" means:

The unstable/fold boundary is below the zero-speed-acceleration level. Therefore, the chosen upper branch includes the relevant $\dot{V} = 0$ crossing and has the correct sign for velocity stabilization.

If the separatrix rose above the $\dot{V} = 0$ plane, then the useful stable crossing might be lost or the branch selection might no longer guarantee stable speed dynamics.

Goh numerically checks this over a broad state-space range and reports that the condition is generally valid except for very high yaw rates at very low speeds. Goh_AutomatedVehicleControl

So the speed stability claim is not a global theorem for all possible states. It is a physically motivated and numerically checked local/operating-region property.

14. Projection into feasible space

The controller computes:

$$(\dot{\phi}_{des}, \dot{r}_{des})$$

But the vehicle may not be able to achieve that pair.

For example, the desired pair might lie outside the feasible projection of the derivative surface onto the:

$$(\dot{\phi}, \dot{r})$$

plane.

Then Goh must project the command into the feasible set.

In Chapter 2, the projection is deliberately simple:

1. Compute $\dot{\phi}_{des}$ from the path controller.
2. Saturate $\dot{\phi}_{des}$ so it lies within feasible values.
3. Use that saturated $\dot{\phi}_{des}$ to compute r_{syn} and $\dot{r}_{des}$.

4. If the resulting pair is still outside the feasible space, project it along the line:

$$\dot{r} = \dot{r}_{des}$$

That means he keeps the yaw-acceleration command as much as possible and sacrifices course-rate/path tracking first. The thesis describes this as a naive method that roughly prioritizes the unstable yaw/sideslip dynamics over lateral-error dynamics. Goh_AutomatedVehicleControl

The physical reason is:

$$\text{Path error is bad, but losing yaw/sideslip control can cause spin.}$$

So if you cannot do both, it is better to protect yaw/sideslip dynamics.

Chapter 3 later improves this projection because the Chapter 2 projection is not sophisticated enough for highly dynamic Figure-8 transitions.

15. Why wheel dynamics enter after nonlinear inversion

Up to now, the high-level model inversion assumes rear force direction γ can be commanded.

But the real actuator is not γ.

The real actuator is torque:

$$\tau$$

The rear force direction is determined by tire slip, and tire slip depends on wheel speed:

$$\omega$$

So the chain is:

$$\tau \to \omega \to \text{rear slip velocity} \to \gamma \to (F_{xr}, F_{yr}) \to (\dot{\phi}, \dot{r}, \dot{V})$$

If you ignore wheel dynamics and set:

$$\tau = RF_{xr,des}$$

you assume force changes instantaneously. Goh argues this assumption is poor during drifting because rear tires are fully sliding and wheel speed strongly affects the direction of the saturated force vector. The thesis first notes the common simple mapping $\tau = RF_{xr}$, then directly challenges it by modeling saturated rear tire force and wheel dynamics. Goh_AutomatedVehicleControl

16. Saturated rear tire force model

When the rear tire is fully sliding, the tire force magnitude is approximately fixed by friction:

$$F_{xr}^2 + F_{yr}^2 = \mu F_{zr}$$

So the rear force lies on the friction circle:

$$F_{xr}^2 + F_{yr}^2 = (\mu F_{zr})^2$$

The control-relevant variable is the force direction:

$$\gamma = \text{atan2}(F_{yr}, F_{xr})$$

If:

$$F_{xr} = \mu F_{zr} \cos\gamma$$

$$F_{yr} = \mu F_{zr} \sin\gamma$$

then changing γ rotates the rear force vector around the friction circle.

Goh models the saturated tire force as opposing the slip velocity vector. At full sliding, changing wheel speed changes the longitudinal component of slip velocity, so it rotates the total rear force vector. The thesis gives the relationship:

$$\gamma = \arctan\begin{pmatrix} -(V\sin\beta - br) \\ -V\cos\beta + R\omega \end{pmatrix}$$

where R is tire radius and ω is wheel speed. Goh_AutomatedVehicleControl

This equation says:

$$\omega \text{ controls } \gamma.$$

Not directly, instantaneously, or perfectly — but physically, wheel speed is what sets the slip velocity direction, and the slip velocity direction sets the rear force direction.

17. Mapping γ_{des} to ω_{des}

From:

$$\tan\gamma = \frac{-(V\sin\beta - br)}{-V\cos\beta + R\omega}$$

solve for ω.

Let:

$$A = -(V\sin\beta - br)$$

Then:

$$\tan\gamma = \frac{A}{-V\cos\beta + R\omega}$$

So:

$$-V\cos\beta + R\omega = \frac{A}{\tan\gamma}$$

$$R\omega = V\cos\beta + \frac{A}{\tan\gamma}$$

Therefore:

$$\omega_{des} = \frac{1}{R}\left[V\cos\beta + \frac{-(V\sin\beta - br)}{\tan\gamma_{des}}\right]$$

This is the tire-slip mapping stage:

$$\gamma_{des} \to \omega_{des}$$

The high-level controller says:

$$\text{I want rear force direction } \gamma_{des}.$$

The tire-slip mapping says:

$$\text{Then the rear wheel speed should be } \omega_{des}.$$

The low-level wheelspeed loop says:

$$\text{Use torque } \tau \text{ to make } \omega \to \omega_{des}.$$

18. Wheel dynamics

The wheel rotational dynamics are:

$$I_\omega\dot{\omega} = \tau - RF_{xr}$$

where:

$$I_\omega = \text{wheel} + \text{drivetrain rotational inertia}$$

$$\tau = \text{applied drive torque}$$

$$RF_{xr} = \text{resisting tire force torque}$$

The thesis gives this as the wheel-speed dynamic equation. Goh_AutomatedVehicleControl

This means torque does not directly equal tire force during transients.

If you suddenly increase torque, first:

$$\dot{\omega} \text{ changes}$$

then:

$$\omega \text{ changes}$$

then:

$$\gamma \text{ changes}$$

then:

$$F_{xr}, F_{yr} \text{ change}$$

then:

$$\dot{r}, \dot{\phi}, \dot{V} \text{ change}$$

So if the high-level controller assumes force is immediate, the real vehicle will have delay and phase lag.

That is exactly what Goh later observes experimentally when wheelspeed control is disabled: measured force versus commanded force makes loops, indicating phase lag; with wheelspeed control, measured force is much closer to the ideal one-to-one relationship. Goh_AutomatedVehicleControl

19. Why controlling γ is better than directly controlling F_{xr}

Goh gives two reasons.

Reason 1: robustness to μF_z uncertainty

Suppose you try to hold F_{xr} constant.

The rear tire is on the friction circle:

$$F_{yr} = \quad (\mu F_z)^2 - F_{xr}^2$$

If μF_z changes slightly, F_{yr} can change a lot, especially when F_{xr} is large.

But if you hold γ constant, then:

$$F_{xr} = \mu F_z \cos\gamma$$

$$F_{yr} = \mu F_z \sin\gamma$$

Now changes in μF_z scale both forces more predictably.

Goh writes the sensitivity comparison as:

$$\frac{\partial F_{yr}}{\partial(\mu F_{zr})}_{F_{xr}} = \frac{1}{\sin\gamma}$$

$$\left.\frac{\partial F_{xr}}{\partial(\mu F_{zr})}\right|_{F_{xr}} = 0$$

but for constant γ:

$$\left.\frac{\partial F_{yr}}{\partial(\mu F_{zr})}\right|_{\gamma} = \sin\gamma$$

$$\left.\frac{\partial F_{xr}}{\partial(\mu F_{zr})}\right|_{\gamma} = \cos\gamma$$

The thesis explains that holding F_{xr} can make F_{yr} highly sensitive to μF_z, while holding γ gives smaller, more proportional changes. Goh_AutomatedVehicleControl

So the controller prefers to think in terms of rear force direction γ, then implement that through wheel speed.

Reason 2: force-generation delay is significant

Goh shows that rear force rise time due to wheel dynamics can be large. He compares rear sliding-tire force response with a front tire relaxation-length estimate. At the cited drift condition, the rear force rise time around the equilibrium force is more than five times the front lateral force rise time. The thesis says this delay matters because the controller coordinates steering and rear force over a broad range of conditions. Goh_AutomatedVehicleControl

So the wheelspeed controller is needed to make the force-based high-level inversion valid.

The logic is:

$$\text{High-level inversion assumes rear force is a usable input.}$$

$$\text{But torque does not instantly create rear force.}$$

$$\text{So close a low-level wheel-speed loop.}$$

$$\text{Then rear force direction behaves fast enough to be treated as a high-level input.}$$

20. The wheelspeed control law

Goh uses:

$$\tau_{command} = -k_\omega I_\omega(\omega - \omega_{des,f}) + I_\omega\dot{\omega}_{des,f} + RF_{xr,des}$$

with:

$$\dot{\omega}_{des,f} = -\frac{1}{t_\omega}(\omega_{des,f} - \omega_{des})$$

Here:

$$\omega_{des} = \text{raw desired wheel speed from } \gamma_{des}$$

$$\omega_{des,f} = \text{filtered desired wheel speed}$$

$$RF_{xr,des} = \text{feedforward torque needed to support desired longitudinal force}$$

$$-k_\omega I_\omega(\omega - \omega_{des,f}) = \text{feedback torque correcting wheel-speed error}$$

$$I_\omega\dot{\omega}_{des,f} = \text{feedforward torque for desired wheel acceleration}$$

The thesis says this maps the rear longitudinal force/thrust-angle command to desired wheel speed, then uses the above torque command to achieve it with acceptable bandwidth. Goh_AutomatedVehicleControl

To see why this works, substitute into wheel dynamics:

$$I_\omega\dot{\omega} = \tau - RF_{xr}$$

Assume the feedforward tire force is accurate:

$$F_{xr} \approx F_{xr,des}$$

Then:

$$I_\omega\dot{\omega} = -k_\omega I_\omega(\omega - \omega_{des,f}) + I_\omega\dot{\omega}_{des,f} + RF_{xr,des} - RF_{xr}$$

Cancel approximately:

$$RF_{xr,des} - RF_{xr} \approx 0$$

So:

$$\dot{\omega} = -k_\omega(\omega - \omega_{des,f}) + \dot{\omega}_{des,f}$$

Define:

$$e_\omega = \omega - \omega_{des,f}$$

Then:

$$\dot{e}_\omega = \dot{\omega} - \dot{\omega}_{des,f} = -k_\omega e_\omega$$

So wheel-speed error decays exponentially:

$$e_\omega(t) \to 0$$

That means:

$$\omega \to \omega_{des,f}$$

and therefore:

$$\gamma \to \gamma_{des}$$

approximately, at a designed bandwidth.

21. Why the desired wheel speed is filtered

The desired wheel speed ω_{des} can change sharply because γ_{des} comes from a nonlinear inversion.

If the controller directly commanded sharp ω_{des}, the required torque could be huge or noisy.

So Goh uses:

$$\dot{\omega}_{des,f} = -\frac{1}{t_\omega}(\omega_{des,f} - \omega_{des})$$

This is a first-order filter.

It smooths the desired wheel-speed command and makes $\dot{\omega}_{des,f}$ available for feedforward acceleration.

So the low-level loop is not:

$$\omega \to \omega_{des}$$

instantaneously.

It is:

$$\omega \to \omega_{des,f}$$

where $\omega_{des,f}$ is a realizable filtered command.

22. How the whole Chapter 2.2–2.3 controller runs online

At each control update:

1. Measure or estimate:
$$r, \quad V, \quad \beta, \quad \omega, \quad e, \quad \Delta\phi$$
2. Read reference path quantities at current s:
$$\kappa_{ref}, \quad \phi_{ref}, \quad \beta_{ref}, \quad \dot{\beta}_{ref}, \quad r_{ref}, \quad \dot{r}_{ref}$$
3. Compute desired course-rate for path tracking:
$$\dot{\phi}_{des} = -\frac{k_p}{V}e - k_d\Delta\phi + \dot{\phi}_{ref}$$
4. Compute desired sideslip rate:
$$\dot{\beta}_{des} = -k_\beta(\beta - \beta_{ref}) + \dot{\beta}_{ref}$$
5. Compute synthetic yaw rate:
$$r_{syn} = \dot{\phi}_{des} - \dot{\beta}_{des}$$
6. Compute desired yaw acceleration:
$$\dot{r}_{des} = -k_r(r - r_{syn}) + \dot{r}_{syn}$$
7. Check whether:
$$(\dot{\phi}_{des}, \dot{r}_{des})$$
is feasible.
8. If not feasible, project it into feasible space.
9. Numerically invert the nonlinear force-based vehicle model:
$$(\dot{\phi}_{des}, \dot{r}_{des}) \mapsto (\delta, \gamma_{des})$$
on the desired branch of the feasible derivative surface.
10. Convert γ_{des} to ω_{des} using the saturated rear tire slip geometry.
11. Filter ω_{des} to get $\omega_{des,f}$.
12. Command torque:
$$\tau_{command} = -k_\omega I_\omega(\omega - \omega_{des,f}) + I_\omega\dot{\omega}_{des,f} + RF_{xr,des}$$
13. The real vehicle receives:
$$\delta, \quad \tau$$
and the loop repeats.

23. The deepest conceptual idea

The deepest idea in Chapter 2.2–2.3 is this:

In normal driving:

$$\delta \text{ tracks path}, \quad \tau \text{ tracks speed}, \quad \beta \text{ is naturally stable.}$$

In drifting:

$$\delta \text{ and } \tau \text{ together track path and stabilize } \beta, \quad V \text{ becomes stable through coupling.}$$

So the controller swaps the "free stable thing."

Normal driving:

$$\beta \text{ is free but stable}$$

Drifting:

$$V \text{ is free but stable}$$

That is why Goh's velocity zero-dynamics argument is central. Without it, the controller would be incomplete, because it would be spending both inputs on path and sideslip while leaving speed uncontrolled. The thesis explicitly frames this as using the coupling between state dynamics to yield a stable velocity response, instead of fighting that coupling. Goh_AutomatedVehicleControl

24. Why Chapter 2.3 is not a small detail

Chapter 2.2 gives a beautiful force-level controller:

$$(\dot{\phi}_{des}, \dot{r}_{des}) \to (\delta, \gamma_{des})$$

But the real car does not actuate γ. It actuates torque.

If wheel dynamics are slow, then the high-level controller thinks it achieved a rear force direction, but the vehicle has not actually achieved it yet. That creates phase lag in yaw/sideslip dynamics, which is very dangerous near unstable drift states.

So Chapter 2.3 is the bridge between:

$$\text{elegant nonlinear force-level control}$$

and

$$\text{real drivetrain-limited experimental control}$$

That is why the experimental comparison without wheelspeed control is important: it shows larger oscillations and force phase lag when the inner wheel-speed loop is removed. Goh_AutomatedVehicleControl

25. How this connects to your own high-fidelity vehicle model work

For your own vehicle simulation / direct optimal control / MPC work, the lesson is not that you should copy Goh's single-track model. Your model is much higher fidelity.

The real lesson is structural:

$$\text{First decide which motion derivatives you really need.}$$

Then:

$$\text{Use model inversion or optimization to find actuator commands that achieve them.}$$

Then:

$$\text{Check which internal dynamics are left uncontrolled and whether they are stable.}$$

In Goh:

$$\text{controlled derivatives} = (\dot{\phi}, \dot{r})$$

$$\text{inputs} = (\delta, \gamma)$$

$$\text{uncontrolled derivative} = \dot{V}$$

$$\text{internal zero dynamic} = V$$

In your 14-DOF / 3D ribbon-road case, the equivalent question would be much larger:

$$\text{Which outputs do you track?}$$

$$\text{Which actuator combinations produce the required generalized accelerations?}$$

$$\text{Which internal dynamics are left free?}$$

$$\text{Are those internal dynamics stable or at least bounded?}$$

That is why Goh is important: he does not just "make a drift controller." He shows how to deliberately choose controlled outputs, exploit coupled dynamics, use nonlinear inversion, manage feasibility, and close a lower-level actuator loop so the high-level force assumptions become valid.

資料來源

Double check your answers, make sure they are both physically and mathematically correct and consistent with Goh's thesis. Then give me the most detailed and correct full version of explanations of Chapter 2 and Chapter 3 so that I can learn these two chapters with the materials you give to me.

已思考 2m 36s

Below is the corrected, full learning version. I will be careful about three things that are easy to misunderstand:

1. **Goh's controller is not a normal path/speed controller.** It controls **path position** and **sideslip**, while speed V is left untracked but argued to be stable as a zero dynamic.
2. **The "state derivatives" are not arbitrary mathematical objects.** They are the physically achievable instantaneous accelerations/rates of the vehicle: mainly $\dot{\phi}$, $\dot{r}$, and $\dot{V}$.
3. **The rear thrust angle γ should be interpreted as the angle of the saturated rear tire force vector**, so the physically consistent parameterization is

$$F_{xr} = \mu F_{zr} \cos\gamma, \qquad F_{yr} = \mu F_{zr} \sin\gamma, \qquad \gamma = \operatorname{atan2}(F_{yr}, F_{xr}).$$

This is the interpretation used by the wheel-speed and Chapter 3 planning equations, even though one line in the thesis text can look ambiguous when read in isolation. The later equations make the intended meaning clear. Goh_AutomatedVehicleControl

Chapter 2 — Controller Design for Drifting Along a General Path

1. What Chapter 2 is really trying to do

Chapter 2 solves this problem:

$$\text{Make an autonomous vehicle drift along a general path while tracking sideslip.}$$

This is much harder than normal path tracking.

In normal driving, the usual idea is:

$$\delta \to \text{path tracking}, \qquad \tau \to \text{speed tracking}, \qquad \beta \approx 0 \text{ and naturally stable.}$$

Here:

$$\delta = \text{steering angle}, \qquad \tau = \text{drive torque}, \qquad \beta = \text{sideslip}.$$

But in drifting, sideslip is large and unstable. The rear tires are saturated, and throttle changes the rear force direction, which affects yaw. Steering also affects speed because the vehicle is moving at a large angle relative to its body. Therefore, the normal separation

$$\text{steering for path, throttle for speed}$$

is no longer valid.

Goh's key conceptual change is:

$$\delta \text{ and throttle are used together to track path and sideslip.}$$

Then speed V is **not explicitly tracked**. Instead, Goh argues that speed becomes a stable internal/zero dynamic because of the coupling between lateral and longitudinal dynamics at high sideslip. This is stated as one of the main results of Chapter 2. Goh_AutomatedVehicleControl

So Chapter 2 is really a controlled "reshuffling" of the vehicle-control tasks:

Normal driving	Drifting in Goh Chapter 2
Steering tracks path	Steering + rear force jointly track path
Throttle tracks speed	Steering + rear force jointly stabilize sideslip
Sideslip is untracked but stable	Speed is untracked but stable

That last line is the deepest point.

2. Vehicle states and path variables

Goh uses a planar single-track model. The main dynamic states are

$$r = \text{yaw rate},$$

$$V = \text{magnitude of vehicle CG velocity},$$

$$\beta = \text{sideslip angle}.$$

The vehicle also has a body heading ψ, but the path is not tracked using ψ. Instead, Goh defines the **course angle**:

$$\phi = \text{direction of the velocity vector in the inertial frame}.$$

This distinction is absolutely central.

In normal driving:

$$\phi \approx \psi.$$

In drifting:

$$\phi \not\approx \psi.$$

The car body can point far away from the direction the vehicle is actually moving. Therefore, if you want the vehicle to follow a path, you should control the direction of the **velocity vector**, not the body heading.

The key kinematic relation is

$$\dot{\phi} = \dot{\beta} + r.$$

Equivalently,

$$\dot{\beta} = \dot{\phi} - r.$$

This equation is the heart of Goh's controller. It says:

- $\dot{\phi}$ is how fast the **velocity vector** rotates.
- r is how fast the **vehicle body** rotates.
- Their difference determines how sideslip changes.

So Goh uses:

$$\dot{\phi} \quad \text{for path tracking},$$

and

$$r, \dot{r} \quad \text{for sideslip stabilization.}$$

The thesis explicitly defines ϕ as the velocity-vector direction, $\dot{\phi}$ as the course rate, and gives $\dot{\phi} = \dot{\beta} + r$. Goh_AutomatedVehicleControl

3. Path tracking dynamics

The vehicle is described relative to a reference path.

Let

$$e = \text{lateral error from the path},$$

$$s = \text{distance along the path},$$

$$\phi_{\text{ref}}(s) = \text{reference path tangent angle},$$

$$\Delta\phi = \phi - \phi_{\text{ref}}.$$

Then the lateral error dynamics are

$$\dot{e} = V \sin(\Delta\phi).$$

If the course error $\Delta\phi$ is small,

$$\dot{e} \approx V \Delta\phi.$$

Differentiating,

$$\ddot{e} \approx V\Delta\dot{\phi} + \dot{V}\Delta\phi.$$

Goh then uses

$$V\Delta\dot{\phi} \gg \dot{V}\Delta\phi$$

during closed-loop tracking, so

$$\ddot{e} \approx V\Delta\dot{\phi}.$$

Important: this is **not** a small-sideslip assumption. It is a small **course-error** assumption. The vehicle may have -40° sideslip, but if the velocity vector is close to tangent to the path, $\Delta\phi$ is small. Goh later validates this approximation experimentally by comparing the two contributions to $\ddot{e}$. Goh_AutomatedVehicleControl

The reference course-rate is

$$\dot{\phi}_{\text{ref}} = \kappa_{\text{ref}} \frac{V\cos\Delta\phi}{1 - \kappa_{\text{ref}} e},$$

where

$$\kappa_{\text{ref}} = \frac{d\phi_{\text{ref}}}{ds}.$$

If

$$e = 0, \qquad \Delta\phi = 0,$$

then

$$\dot{\phi}_{\text{ref}} = V\kappa_{\text{ref}}.$$

This simple relation becomes very important for the velocity-zero-dynamics argument.

4. Vehicle force model

Goh's force-based single-track model uses front and rear tire forces:

$$F_{yf}, F_{xf}, F_{yr}, F_{xr}.$$

For MARTY, a rear-wheel-drive vehicle,

$$F_{xf} = 0.$$

The dynamic equations are

$$\dot{r} = \frac{aF_{yf}\cos\delta + aF_{xf}\sin\delta - bF_{yr}}{I_z},$$

$$\dot{\beta} = \frac{F_{yf}\cos(\delta-\beta) + F_{xf}\sin(\delta-\beta) + F_{yr}\cos\beta - F_{xr}\sin\beta}{mV} - r,$$

$$\dot{V} = \frac{-F_{yf}\sin(\delta-\beta) + F_{xf}\cos(\delta-\beta) + F_{yr}\sin\beta + F_{xr}\cos\beta}{m}.$$

These equations show the coupling.

At large β, F_{xr} appears inside the $\dot{\beta}$ equation through

$$-F_{xr}\sin\beta,$$

so throttle affects sideslip/yaw behavior.

Also, F_{yf} appears inside $\dot{V}$ through

$$-F_{yf}\sin(\delta-\beta),$$

so steering affects speed.

This is why Goh cannot use the conventional structure "steering controls path, throttle controls speed." The single-track model and its equations of motion are introduced in Section 2.1.1. Goh_AutomatedVehicleControl

5. Tire models

5.1 Front tire

The front tire is modeled with a Fiala brush model. The front tire is not assumed to be fully sliding all the time. It may be near saturation, but its force is still computed as a nonlinear function of slip angle.

The important point is:

$$F_{yf} = F_{yf}(\alpha_f, F_z, \mu, C_\alpha).$$

So steering δ affects front slip angle, which affects F_{yf}, which affects both $\dot{r}$ and $\dot{V}$.

5.2 Rear tire

During drifting, the rear tire is assumed fully saturated:

$$F_{xr}^2 + F_{yr}^2 = (\mu F_{zr})^2.$$

So the rear tire force vector lies on the friction circle.

Therefore, one convenient representation is

$$F_{xr} = \mu F_{zr}\cos\gamma,$$

$$F_{yr} = \mu F_{zr}\sin\gamma.$$

Here γ is the rear thrust angle / rear force-vector angle.

This means rear drive torque does not simply "add force." Since the rear tire is already saturated, changing longitudinal slip changes the **direction** of the rear force vector. That rotated force vector changes yaw and sideslip.

That is why the controller later wants to command γ, not just F_{xr}. Goh introduces the rear friction-circle assumption in Section 2.1.3. Goh_AutomatedVehicleControl

6. Chapter 2 controller architecture

Goh's controller is a cascade:

$$(e, \Delta\phi, \beta, r, V) \to (\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}) \to (\delta, \gamma_{\text{des}}) \to \omega_{\text{des}} \to \tau.$$

In words:

1. Use path and sideslip errors to decide what motion derivatives the vehicle should have.
2. Use nonlinear model inversion to find steering and rear thrust angle that produce those derivatives.
3. Convert desired rear thrust angle into desired rear wheel speed.
4. Use torque to track desired wheel speed.

Goh's Figure 2.2 explicitly shows this sequence:

$$\text{Imposed error dynamics} \to \text{nonlinear model inversion} \to \text{tire slip mapping} \to \text{wheelspeed control loop} \to \text{vehicle dynamics}.$$

It also labels V as "free, but stable." Goh_AutomatedVehicleControl

7. Stage 1: impose path-tracking error dynamics

Goh first imposes stable second-order dynamics on lateral error:

$$\ddot{e}_{\text{des}} = -k_p e - k_d \dot{e}.$$

Using

$$\dot{e} \approx V\Delta\phi,$$

$$\ddot{e} \approx V\Delta\dot{\phi},$$

he gets

$$V\Delta\dot{\phi}_{\text{des}} = -k_p e - k_d V\Delta\phi.$$

Divide by V:

$$\Delta\dot{\phi}_{\text{des}} = -\frac{k_p}{V} e - k_d\Delta\phi.$$

Then add the reference course-rate:

$$\dot{\phi}_{\text{des}} = -\frac{k_p}{V} e - k_d\Delta\phi + \dot{\phi}_{\text{ref}}.$$

This is the first desired state derivative.

Physical meaning:

- If $e > 0$, rotate the velocity vector back toward the path.
- If $\Delta\phi \neq 0$, align the velocity vector with the path tangent.
- If errors are zero, follow the path curvature through $\dot{\phi}_{\text{ref}}$.

This is the first major insight:

> Path tracking during drifting should control the velocity vector, not the body heading.

Goh derives this as Equations 2.8–2.9. Goh_AutomatedVehicleControl

8. Stage 2: impose sideslip dynamics using yaw rate as a synthetic input

The sideslip relation is

$$\dot{\beta} = \dot{\phi} - r.$$

Since $\dot{\phi}_{\text{des}}$ has already been chosen for path tracking, the remaining way to control $\dot{\beta}$ is to control r relative to $\dot{\phi}$.

Goh imposes first-order sideslip error dynamics:

$$e_\beta = \beta - \beta_{\text{ref}},$$

$$\dot{e}_\beta = -k_\beta e_\beta.$$

Therefore,

$$\dot{\beta}_{\text{des}} = -k_\beta e_\beta + \dot{\beta}_{\text{ref}}.$$

Now use

$$\dot{\beta}_{\text{des}} = \dot{\phi}_{\text{des}} - r_{\text{syn}}.$$

Solving for the yaw rate that would achieve the desired sideslip dynamics gives

$$r_{\text{syn}} = \dot{\phi}_{\text{des}} - \dot{\beta}_{\text{des}}.$$

Substitute $\dot{\beta}_{\text{des}}$:

$$r_{\text{syn}} = \dot{\phi}_{\text{des}} + k_\beta e_\beta - \dot{\beta}_{\text{ref}}.$$

This r_{syn} is not a physical actuator. It is a **synthetic yaw-rate target**.

Then Goh imposes first-order tracking of r_{syn}:

$$e_r = r - r_{\text{syn}},$$

$$\dot{e}_r = -k_r e_r.$$

Therefore,

$$\dot{r} - \dot{r}_{\text{syn}} = -k_r(r - r_{\text{syn}}),$$

so

$$\dot{r}_{\text{des}} = -k_r(r - r_{\text{syn}}) + \dot{r}_{\text{syn}}.$$

This is the second desired state derivative.

At zero error,

$$e = 0, \quad \Delta\phi = 0, \quad e_\beta = 0, \quad e_r = 0,$$

Goh's construction gives

$$\dot{r}_{\text{des}} = \dot{r}_{\text{ref}}.$$

Goh gives these relations in Equations 2.10–2.13. Goh_AutomatedVehicleControl

9. What "state derivatives" means

The controller outputs

$$\dot{\phi}_{\text{des}} \quad \text{and} \quad \dot{r}_{\text{des}}.$$

These are called desired state derivatives.

They are not actuator commands. The vehicle cannot directly accept $\dot{\phi}$ and $\dot{r}$.

They mean:

$$\dot{\phi}_{\text{des}} = \text{desired instantaneous rotation rate of the velocity vector},$$

$$\dot{r}_{\text{des}} = \text{desired yaw acceleration}.$$

So the controller first asks:

$$\text{What instantaneous motion should the car have?}$$

Only after that does it ask:

$$\text{What steering and rear force direction produce that motion?}$$

This is why nonlinear inversion is needed.

10. Stage 3: nonlinear model inversion

The force-based vehicle model gives a mapping

$$(\delta, \gamma) \mapsto (\dot{\phi}, \dot{r}, \dot{V}).$$

At a fixed state

$$x = (r, V, \beta),$$

and with fixed model parameters, every steering angle δ and rear thrust angle γ produce some instantaneous state derivatives.

Goh wants

$$\dot{\phi} = \dot{\phi}_{\text{des}},$$

$$\dot{r} = \dot{r}_{\text{des}}.$$

So he numerically inverts the model:

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}) \to (\delta, \gamma_{\text{des}}).$$

This is nonlinear because:

- the model contains $\sin$, $\cos$, $\tan$,
- sideslip is large,
- steering is not small,
- the front tire force is nonlinear,
- the rear tire is constrained by the friction circle,
- rear force direction and yaw moment are coupled.

Goh explicitly says the force-based nonlinear single-track model is numerically inverted to map $(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}})$ into steering angle δ and rear thrust angle γ. Goh_AutomatedVehicleControl

11. Why speed V is not directly controlled

This is where the controller becomes conceptually unusual.

The vehicle has two main inputs:

$$\delta, \qquad \gamma.$$

Goh uses those two inputs to command two desired derivatives:

$$\dot{\phi}_{\text{des}}, \qquad \dot{r}_{\text{des}}.$$

Therefore, there is no remaining independent input to impose

$$\dot{V}_{\text{des}}.$$

So

$$V \text{ is not explicitly regulated.}$$

This is not an oversight. It is a deliberate design choice.

Why? Because Goh chooses the two most safety-critical objectives in drifting:

$$\text{path tracking} \quad \text{and} \quad \text{sideslip stabilization}.$$

Speed is left as an internal dynamic.

The key question is then:

$$\text{Will } V \text{ remain stable, or will it drift away?}$$

This is where zero dynamics enters.

12. What zero dynamics means here

In nonlinear control, the "zero dynamics" are the internal dynamics that remain when the controlled output errors are zero.

Here the controller tries to drive

$$e \to 0,$$

$$\Delta\phi \to 0,$$

$$e_\beta \to 0,$$

$$e_r \to 0.$$

When these are zero, what remains unconstrained?

$$V.$$

So Goh studies V as the zero dynamic.

At zero path/course error,

$$\dot{\phi}_{\text{des}} = \dot{\phi}_{\text{ref}}.$$

Using the path relation,

$$\dot{\phi}_{\text{ref}} = V\kappa_{\text{ref}}\frac{\cos\Delta\phi}{1 - e\kappa_{\text{ref}}}.$$

At

$$e = 0, \qquad \Delta\phi = 0,$$

this becomes

$$\dot{\phi}_{\text{des}} = V\kappa_{\text{ref}}.$$

Also, at zero yaw/sideslip error,

$$\dot{r}_{\text{des}} = \dot{r}_{\text{ref}}.$$

These are exactly Equations 2.14–2.17. Goh_AutomatedVehicleControl

So under perfect tracking, the desired course rate depends linearly on speed:

$$\dot{\phi}_{\text{des}} = V\kappa_{\text{ref}}.$$

Now imagine V becomes too high. Then the demanded $\dot{\phi}_{\text{des}}$ becomes larger. If, on the selected branch of the dynamics,

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

along constant $\dot{r}$, then increasing $\dot{\phi}$ makes $\dot{V}$ smaller. That slows the car down.

Similarly, if V becomes too low, $\dot{\phi}_{\text{des}}$ becomes smaller, and the coupling tends to make $\dot{V}$ positive. That speeds the car up.

So the restoring mechanism is:

$$V \uparrow \Rightarrow \dot{\phi}_{\text{des}} \uparrow \Rightarrow \dot{V} < 0 \Rightarrow V \downarrow .$$

$$V \downarrow \Rightarrow \dot{\phi}_{\text{des}} \downarrow \Rightarrow \dot{V} > 0 \Rightarrow V \uparrow .$$

This is why Goh needs two conditions:

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

and

$$\dot{V} \text{ has a zero crossing along constant } \dot{r}.$$

The zero crossing gives an equilibrium speed. The negative slope gives local stability. Goh states this condition directly after deriving the zero-error relation $\dot{\phi}_{\text{des}} = V\kappa_{\text{ref}}$. Goh_AutomatedVehicleControl

13. The feasible surface of state derivatives

At a fixed vehicle state

$$(r, V, \beta),$$

vary the inputs

$$\delta, \gamma.$$

For every input pair, the model produces

$$(\dot{\phi}, \dot{r}, \dot{V}).$$

Because there are two inputs, the achievable set is a two-dimensional surface in three-dimensional derivative space:

$$(\delta, \gamma) \mapsto (\dot{\phi}, \dot{r}, \dot{V}).$$

This is the **surface of possible state derivatives**.

Goh's Figures 2.3–2.5 visualize this surface. The controller wants to choose a point on this surface whose $(\dot{\phi}, \dot{r})$ coordinates match the desired values. The corresponding $\dot{V}$ is then whatever the surface gives.

So the model inversion can be visualized as:

$$\text{desired point in } (\dot{\phi}, \dot{r}) \rightarrow \text{point on feasible derivative surface} \rightarrow (\delta, \gamma).$$

The shape of this surface determines whether V is stable or unstable as an internal dynamic.
Goh_AutomatedVehicleControl

14. Why there can be more than one inverse solution

The projection of the derivative surface onto the $(\dot{\phi}, \dot{r})$ plane can fold over itself.

That means the same pair

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}})$$

may correspond to two different actuator pairs

$$(\delta_1, \gamma_1), \qquad (\delta_2, \gamma_2),$$

with different $\dot{V}$.

One branch may have

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

and give stable speed behavior.

Another branch may have

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} > 0$$

and give unstable speed behavior.

Therefore, Goh must choose the correct branch.

This is where the separatrix enters.

15. The separatrix in Chapter 2

The separatrix is the fold boundary separating the useful branch of the derivative surface from the other branch.

Mathematically, it occurs where the mapping from inputs to $(\dot{\phi}, \dot{r})$ becomes locally singular.

The determinant condition is

$$\frac{\partial \dot{\phi}}{\partial \delta}\frac{\partial \dot{r}}{\partial F_{xr}} - \frac{\partial \dot{\phi}}{\partial F_{xr}}\frac{\partial \dot{r}}{\partial \delta} = 0.$$

This is Equation 2.18. Goh says that as long as this separatrix lies below the plane

$$\dot{V} = 0,$$

then constraining the model inversion to the "upper" surface should give stable velocity zero dynamics. It also makes the inverse mapping one-to-one, simplifying numerical inversion. Goh_AutomatedVehicleControl

Physical meaning:

- The separatrix is the boundary where the inverse becomes ambiguous.
- Above it, the desired slope

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

holds.

- If the $\dot{V} = 0$ crossing lies on this upper branch, speed can self-stabilize.
- By using only this branch, the controller avoids accidentally choosing an actuator solution that gives the wrong speed feedback sign.

Goh numerically checks this over a broad range:

$$0.2 < r < 1.25 \text{ rad/s},$$

$$3 < V < 30 \text{ m/s},$$

$$-5^\circ > \beta > -50^\circ,$$

and finds the condition is valid except at very high yaw rates and very low speeds. Goh_AutomatedVehicleControl

So the velocity-stability claim is not a universal proof for every possible vehicle state. It is a physically motivated local/operating-region result supported by numerical checking and later by experiment.

16. Projection into feasible space in Chapter 2

Sometimes the controller asks for

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}})$$

that the vehicle cannot physically produce.

That means the desired point lies outside the feasible projection of the derivative surface onto the $(\dot{\phi}, \dot{r})$ plane.

In Chapter 2, Goh handles this in a simple way:

1. Compute $\dot{\phi}_{\text{des}}$ from the path controller.
2. Saturate $\dot{\phi}_{\text{des}}$ to feasible values.
3. Use the saturated $\dot{\phi}_{\text{des}}$ to compute r_{syn} and $\dot{r}_{\text{des}}$.
4. If the final pair is still infeasible, project it into the feasible region along the line

$$\dot{r} = \dot{r}_{\text{des}}.$$

This roughly prioritizes yaw/sideslip dynamics over path tracking.

The idea is:

$$\text{Path error can be recovered later.}$$

$$\text{Loss of yaw/sideslip control can cause a spin.}$$

Goh calls this projection naive, and Chapter 3 later improves it. Goh_AutomatedVehicleControl

17. Why Chapter 2.3 adds wheel-speed dynamics

Up to this point, the high-level controller assumes it can command rear thrust angle γ.

But the real actuator is torque:

$$\tau.$$

Torque does not instantaneously create γ. The real chain is:

$$\tau \rightarrow \omega \rightarrow \text{rear slip velocity} \rightarrow \gamma \rightarrow (F_{xr}, F_{yr}) \rightarrow (\dot{\phi}, \dot{r}, \dot{V}).$$

So Goh asks: can we ignore wheel-speed dynamics?

His answer is no, not during drifting.

The rear tire is fully sliding. At full sliding, the tire force vector opposes the slip velocity vector. The slip velocity depends on wheel speed $R\omega$. Therefore, changing wheel speed rotates the rear force vector.

Goh gives the rear force direction relation:

$$\gamma = \arctan\begin{pmatrix} -(V\sin\beta - br) \\ -V\cos\beta + R\omega \end{pmatrix}.$$

A more robust implementation should use atan2 to preserve quadrants.

Solving this for ω gives

$$\omega_{\text{des}} = \frac{1}{R}\left[V\cos\beta + \frac{-(V\sin\beta - br)}{\tan\gamma_{\text{des}}}\right].$$

So the high-level command

$$\gamma_{\text{des}}$$

is converted into a wheel-speed command

$$\omega_{\text{des}}.$$

Goh derives the slip-velocity relationship in Section 2.3.1. Goh_AutomatedVehicleControl

18. Wheel-speed dynamics

The wheel dynamics are

$$I_\omega \dot{\omega} = \tau - RF_{xr}.$$

So torque changes wheel acceleration, not force instantaneously.

Rearrange:

$$\tau = I_\omega \dot{\omega} + RF_{xr}.$$

This is why the wheel-speed control law contains a feedforward term $RF_{xr,\text{des}}$.

Goh's wheel-speed controller is

$$\tau_{\text{command}} = -k_\omega I_\omega(\omega - \omega_{\text{des},f}) + I_\omega \dot{\omega}_{\text{des},f} + RF_{xr,\text{des}}.$$

The filtered desired wheel speed obeys

$$\dot{\omega}_{\text{des},f} = -\frac{1}{t_\omega}(\omega_{\text{des},f} - \omega_{\text{des}}).$$

The logic is simple.

If $F_{xr} \approx F_{xr,\text{des}}$, substitute into the wheel dynamics:

$$I_\omega \dot{\omega} = -k_\omega I_\omega(\omega - \omega_{\text{des},f}) + I_\omega \dot{\omega}_{\text{des},f} + RF_{xr,\text{des}} - RF_{xr}.$$

Cancel the feedforward approximately:

$$I_\omega \dot{\omega} \approx -k_\omega I_\omega(\omega - \omega_{\text{des},f}) + I_\omega \dot{\omega}_{\text{des},f}.$$

Divide by I_ω:

$$\dot{\omega} \approx -k_\omega(\omega - \omega_{\text{des},f}) + \dot{\omega}_{\text{des},f}.$$

Let

$$e_\omega = \omega - \omega_{\text{des},f}.$$

Then

$$\dot{e}_\omega = -k_\omega e_\omega.$$

So the wheel-speed error decays exponentially.

This makes the rear force direction track the high-level thrust-angle command with acceptable bandwidth. Goh gives this controller in Equation 2.24. Goh_AutomatedVehicleControl

19. Why wheel-speed control is better than direct torque mapping

Goh gives two main reasons.

19.1 Robustness to μF_z variation

If you command a fixed longitudinal force F_{xr}, then

$$F_{yr} = \sqrt{(\mu F_{zr})^2 - F_{xr}^2}.$$

So if μF_{zr} changes, F_{yr} can change strongly.

For constant F_{xr},

$$\left.\frac{\partial F_{yr}}{\partial(\mu F_{zr})}\right|_{F_{xr}} = \frac{1}{\sin\gamma}.$$

This becomes large when $|\gamma|$ is small.

But if you command force direction γ, then

$$F_{xr} = \mu F_{zr}\cos\gamma,$$
$$F_{yr} = \mu F_{zr}\sin\gamma.$$

So

$$\left.\frac{\partial F_{yr}}{\partial(\mu F_{zr})}\right|_{\gamma} = \sin\gamma,$$
$$\left.\frac{\partial F_{xr}}{\partial(\mu F_{zr})}\right|_{\gamma} = \cos\gamma.$$

That is much more proportional and physically predictable.

This is why commanding γ through wheel speed is more robust than directly commanding F_{xr}.

19.2 Force-generation delay is significant

Goh simulates the rear sliding-tire force response to torque steps near a drift equilibrium. He finds the rear force rise time can be much slower than the front tire lateral-force rise time. Around the equilibrium rear longitudinal force, the rear force rise time is more than five times a front relaxation-length estimate.

That matters because Goh's high-level controller assumes coordinated steering and rear force inputs. If rear force lags steering, the nonlinear inversion is no longer being realized at the vehicle.

The experimental comparison confirms this: without wheelspeed control, measured rear forces show large Lissajous loops relative to commanded forces, indicating phase lag; with wheelspeed control, the measured force is much closer to the ideal one-to-one line. Goh_AutomatedVehicleControl

20. Chapter 2 experimental meaning

Chapter 2 tests a slowly changing drifting path with:

$$\kappa_{\text{ref}} \in [1/20,\ 1/7]\ \text{m}^{-1},$$

speed about

$$25 \text{ to } 45\ \text{km/h},$$

and sideslip up to

$$-40^\circ.$$

With wheelspeed control, the vehicle tracks well:

$$\text{max lateral deviation} \approx -0.36\ \text{m},$$
$$\text{RMS lateral error} \approx 0.18\ \text{m},$$
$$\text{max sideslip deviation} \approx -6.1^\circ,$$
$$\text{RMS sideslip error} \approx 2.4^\circ.$$

The speed remains close to the quasi-equilibrium values, supporting the claim that velocity is stable under the imposed control law. Without wheel-speed control, the vehicle still drifts but with much larger oscillations and phase lag. Goh_AutomatedVehicleControl

So Chapter 2 proves:

$$\text{A vehicle can drift along a general path by controlling } \dot{\phi} \text{ and } \dot{r}.$$

$$\text{Speed can be left untracked if its zero dynamics are stable.}$$

$$\text{Rear wheel-speed dynamics must be considered for accurate force delivery.}$$

Chapter 3 — Execution of Highly Dynamic Drifting Trajectories

Chapter 2 handled a slowly changing, quasi-equilibrium drifting path.

Chapter 3 asks:

$$\text{Can the car execute rapid drift transitions near the physical limits?}$$

The benchmark is a fully autonomous Figure 8 drift.

This is much harder because the vehicle must rapidly transition between large sideslip states:

$$\beta \approx -0.7\ \text{rad} \quad \rightarrow \quad \beta \approx +0.7\ \text{rad}.$$

The maneuver requires high yaw rate, high yaw acceleration, fast steering slew, and high power. Goh says the main issue is no longer simply open-loop instability; it is the loss of ability to alter vehicle states because of actuation and friction limits at extreme yaw/sideslip. Goh_AutomatedVehicleControl

21. Chapter 3 trajectory planning problem

The Figure 8 consists of:

1. steady-state drift circle,
2. rapid transition,
3. opposite steady-state drift circle,
4. rapid transition back.

The circular drift parts are equilibrium segments.

The hard part is the transition.

Goh formulates the transition as a nonlinear optimization problem.

The extended state is

$$X = \begin{bmatrix} r & V & \beta & E & N & \phi & \delta & F_{xr} \end{bmatrix}^T.$$

Here:

$$E, N = \text{global position},$$
$$\delta = \text{steering angle},$$
$$F_{xr} = \text{rear longitudinal force}.$$

The input is actuator slew rate:

$$U = \begin{bmatrix} \dot{\delta} & \dot{F}_{xr} \end{bmatrix}^T.$$

This is important. Goh includes δ and F_{xr} as states so that their rates can be constrained. This matters because the Figure 8 is not only limited by how much steering or force is available, but also by how quickly they can change.

The problem is spatially discretized into N stages with spacing ds. The total transition length

$$s_{\text{total}} = Nds$$

is optimized. Dynamics are integrated using RK4 after dividing by the spatial rate. Goh defines this formulation in Section 3.1. Goh_AutomatedVehicleControl

22. Endpoint constraints

The start and end of the transition are constrained to be drifting equilibria.

A drifting equilibrium satisfies

$$\dot{r} = 0, \qquad \dot{V} = 0, \qquad \dot{\beta} = 0.$$

The desired initial and final sideslip/curvature values are set:

$$\beta_I = -\beta_F = -0.7 \text{ rad},$$

$$\kappa_I = -\kappa_F = \frac{1}{18} \text{ m}^{-1}.$$

The distance between the initial and final centers of rotation is

$$D = 43 \text{ m}.$$

A nominal clothoid is used to choose the terminal pose and initial guess, but the optimized vehicle trajectory is only constrained to match the endpoints, not to follow the clothoid in the middle.

Goh_AutomatedVehicleControl

23. Actuation, torque, and power constraints

Goh imposes constraints on:

$$\dot{\delta},$$

$$\dot{F}_{xr},$$

$$\delta,$$

$$F_{xr},$$

$$\tau,$$

$$P.$$

He also estimates wheel speed from rear saturation. Since

$$F_{xr} = \mu F_{zr} \cos\gamma,$$

he uses

$$\gamma = -\operatorname{sgn}(\alpha_r) \arccos\left(\frac{F_{xr}}{\mu F_{zr}}\right).$$

Then

$$\omega_k = \frac{1}{R}\begin{bmatrix} -(V\sin\beta - br) \\ \tan\gamma \end{bmatrix} + V\cos\beta\Big].$$

Torque and power are then approximated over each stage. The physically consistent torque balance follows

$$\tau \approx I_w\dot{\omega} + RF_{xr},$$

with

$$P = \tau\omega.$$

Goh's optimization uses conservative torque and power limits so that feedback control still has headroom. Table 3.1 includes, for example,

$$P_{\max} = 250 \text{ kW},$$

$$\tau_{\max} = 4.4 \text{ kNm},$$

$$|\dot{\delta}|_{\max} = 2 \text{ rad/s},$$

$$|\dot{F}_{xr}|_{\max} = 20 \text{ kN/s},$$

$$|\delta|_{\max} = 0.66 \text{ rad}.$$

The nonlinear program and cost are solved using CasADi and IPOPT. Goh_AutomatedVehicleControl

24. Cost function

The cost is

$$J = \sum_{k=1}^{N-1} U_k^T R U_k + \sum_{k=1}^{N} Q\left(|\beta_k - \bar{\beta}| - \Delta\beta^*\right)^2.$$

where

$$\bar{\beta} = \frac{\beta_I + \beta_F}{2},$$

$$\Delta\beta^* = |\beta_I - \bar{\beta}| = |\beta_F - \bar{\beta}|.$$

The first term penalizes actuator slew rates:

$$\dot{\delta}, \qquad \dot{F}_{xr}.$$

So it encourages smooth actuation.

The second term encourages sideslip to stay close to either endpoint value, not in the middle. A perfect step transition from β_I to β_F would have zero cost for that term.

Therefore:

- small Q: smoother, slower transition;
- large Q: faster sideslip switch, higher yaw rate, faster steering sweep;
- chosen $Q = 75$: compromise between aggressiveness and actuator headroom.

Goh explicitly explains this competition and the effect of increasing Q. Goh_AutomatedVehicleControl

25. Why Chapter 3 introduces tangent space

Chapter 2 already had a feasible derivative surface.

Chapter 3 makes this idea central.

At a fixed state

$$(r, V, \beta),$$

and with actuator constraints, the set of all possible state derivatives is called the **tangent space**.

Formally:

$$\mathcal{T}(x) = \left\{(\dot{r}, \dot{\phi}, \dot{V}) : u \in \mathcal{U}\right\}.$$

Here u represents physically admissible inputs.

Because the vehicle has two main control inputs, the tangent space is a two-dimensional manifold in a three-dimensional derivative space:

$$(\dot{r}, \dot{\phi}, \dot{V}).$$

Choosing two derivatives fixes the third.

Goh says this explicitly: in the drifting region, the system is coupled and underactuated, so the tangent space is a two-dimensional manifold in three-dimensional space. Goh_AutomatedVehicleControl

This matters because the Chapter 2 controller commands

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}).$$

Therefore, for tracking control, the important view is the projection of the tangent space onto

$$(\dot{\phi}, \dot{r}).$$

The boundary of this projection tells us which combinations of course-rate and yaw acceleration are physically achievable.

26. Why the Figure 8 must operate near the tangent-space boundary

Goh overlays the planned trajectory on the tangent-space boundary and finds that the trajectory comes very close to the limit.

A natural thought is:

> Why not plan with a larger buffer?

Goh explains why this is not easy.

Some tangent-space boundaries are due to actuator limits. Those can be avoided by not using extreme actuator values.

But one boundary is caused by a **fold** in the tangent surface. Near this fold,

$$\dot{V}$$

changes very rapidly for small changes in

$$(\dot{\phi}, \dot{r}).$$

If you impose a large buffer from this fold in the $(\dot{\phi}, \dot{r})$ plane, you may eliminate access to negative $\dot{V}$. That would mean the vehicle could not decelerate in that region, which is too restrictive for useful dynamic drifting.

So highly dynamic drifting fundamentally requires operating near part of the tangent-space boundary.

This is a very important insight:

> The Figure 8 is not hard merely because the controller is imperfect.
>
> It is hard because the desired maneuver physically lives near derivative limits.

Goh explicitly says it is infeasible to leave a large buffer to these limits without overly restricting the planned trajectory. Goh_AutomatedVehicleControl

27. Loss of yaw authority at high sideslip

Goh then studies how the tangent-space boundary changes with sideslip.

As $|\beta|$ increases beyond about 40°, the feasible derivative set shrinks rapidly.

The reason is steering saturation.

The maximum achievable front slip angle is

$$\alpha_{F,\max} = \arctan\left(\frac{V\sin\beta + ar}{V\cos\beta}\right) + \delta_{\max}.$$

When

$$\alpha_{F,\max} = -\alpha_{f,\text{sat}},$$

the vehicle loses the ability to further change front lateral force in the useful direction.

Physically:

- At high sideslip, the front tire may already be near saturation even with maximum counter-steer.
- Once front lateral force cannot be varied meaningfully, steering loses authority over yaw.
- Then the car cannot recover yaw/sideslip effectively.

Goh shows that for $\beta = -50^\circ$, the tangent space only allows positive $\dot{r}$; no input combination can decrease yaw rate. He concludes that near the counter-steering limit, yaw/sideslip dynamics must be prioritized, otherwise the vehicle can terminally spin out. Goh_AutomatedVehicleControl

This is the main physical insight of Chapter 3.

28. Why Chapter 2's naive projection is not enough

Chapter 2 projected infeasible commands roughly along constant $\dot{r}$.

At first, that sounds like yaw prioritization.

But Chapter 3 points out a subtle problem:

$$\dot{r}_{\text{des}}$$

itself depends on

$$\dot{\phi}_{\text{des}}.$$

Why?

Because

$$r_{\text{syn}} = \dot{\phi}_{\text{des}} + k_\beta e_\beta - \dot{\beta}_{\text{ref}},$$

and

$$\dot{r}_{\text{des}} = -k_r(r - r_{\text{syn}}) + \dot{r}_{\text{syn}}.$$

Therefore, if you change $\dot{\phi}_{\text{des}}$, the correct yaw acceleration demand also changes.

So simply keeping the original $\dot{r}_{\text{des}}$ while changing $\dot{\phi}$ is not truly preserving the yaw/sideslip control law.

This is why Chapter 3 modifies the projection.

29. Yaw/sideslip-prioritizing projection

Goh formalizes the trade-off as:

$$\min \ \dot{\phi}(x,u) - \dot{\phi}_{\text{des}}(x,p)$$

subject to

$$\dot{r}(x,u) = \dot{r}_{\text{des}}\left(x, p, \dot{\phi}_{\text{des}}(x,p)\right),$$

$$u_{\min} \le u \le u_{\max}.$$

Interpretation:

- Try to stay as close as possible to the desired path-tracking course rate.
- But enforce the yaw/sideslip dynamics.
- Respect actuator limits.

Solving this nonlinear optimization online would be expensive and potentially unreliable.

So Goh converts it into a geometry problem.

From the yaw control law,

$$\dot{r}_{\text{des}} = -k_r(r - r_{\text{syn}}) + \dot{r}_{\text{syn}}.$$

Substituting

$$r_{\text{syn}} = \dot{\phi}_{\text{des}} + k_\beta e_\beta - \dot{\beta}_{\text{ref}},$$

Goh obtains an affine relation:

$$\dot{r}_{\text{des}} = \left[k_r(-r + k_\beta e_\beta - \dot{\beta}_{\text{ref}}) + \dot{r}_{\text{syn}}\right] + k_r\dot{\phi}_{\text{des}}.$$

So in the $(\dot{\phi}, \dot{r})$ plane, the yaw/sideslip control law is a line with slope k_r.

If the original desired point is infeasible, project it onto the tangent-space boundary **along this yaw-control-law line**.

That is the yaw-stability-preserving projection.

If the line does not intersect the feasible tangent-space region, Goh holds the actuators at the last solution point, which corresponds to an extremal action to restore the car. Goh_AutomatedVehicleControl

The key idea is:

$$\text{Do not preserve the old } \dot{r} \text{ numerically. Preserve the yaw/sideslip control law.}$$

This is a subtle but critical distinction.

30. Computing the tangent-space boundary

To implement the projection, Goh needs the outer boundary of the feasible $(\dot{\phi}, \dot{r})$ set.

For a left-hand drift, he divides the boundary into four segments.

The top point occurs when

$$F_{yf}\cos\delta$$

is maximized and rear lateral force is minimized. Define

$$\delta^+ = \arg\max_\delta \left(F_{yf}\cos\delta\right).$$

Then the top point occurs at

$$\delta = \delta^+, \qquad F_{yr} = 0, \qquad F_{xr} = \mu F_{zr}.$$

The bottom-right boundary corresponds to the fold/separatrix already discussed in Chapter 2. Goh propagates the separatrix condition through the equations of motion and obtains an explicit relation between γ and steering. The remaining boundaries are constant-input contours.

This boundary computation makes the projection fast enough for real-time use. It avoids solving a full nonlinear program at every control step.

Goh gives the four boundary segments in Equation 3.21 and Figure 3.10. Goh_AutomatedVehicleControl

31. The null case

To test whether yaw-prioritization matters, Goh defines a null case:

$$\text{project along the } \dot{\phi} \text{ axis.}$$

This means keep the original $\dot{r}$ demand and only adjust $\dot{\phi}$.

At first glance, this seems to prioritize yaw.

But it does not preserve the coupling between $\dot{r}_{\text{des}}$ and $\dot{\phi}_{\text{des}}$.

So it is not true yaw/sideslip prioritization.

This null case is important experimentally because it isolates the value of the new projection method.

32. Chapter 3 experiments

Goh runs two experiments:

1. **Yaw-stability-preserving projection active:** vehicle completes two laps of the Figure 8.
2. **Naive projection:** vehicle spins out.

With yaw prioritization active:

$$\text{closed-loop yaw rate peaks around } 2 \text{ rad/s},$$

$$\text{steering sweeps through about } 75^\circ \text{ in less than } 1 \text{ second},$$

$$\text{speed reaches about } 50 \text{ km/h}.$$

Lateral error peaks at around

$$1.3 \text{ m}$$

near transitions, but then reduces to generally below

$$0.4 \text{ m}$$

after the transition. Sideslip overshoot is about

$$5^\circ.$$

This is exactly the intended trade-off: sacrifice some temporary path tracking to preserve yaw/sideslip control authority.

With naive projection, the vehicle initially follows similarly, but after peak yaw rate the sideslip overshoots, the tangent space shrinks, yaw authority collapses, and the vehicle spins out. Goh describes this comparison using Figures 3.16–3.17. Goh_AutomatedVehicleControl

33. The deepest Chapter 3 lesson

Chapter 2 says:

$$\text{Control } \dot{\phi} \text{ for path and } \dot{r} \text{ for sideslip.}$$

Chapter 3 says:

$$\text{When both cannot be achieved, prioritize yaw/sideslip authority over path tracking.}$$

This is the key step from ordinary drifting to highly dynamic drifting.

Chapter 2's controller works when the demanded derivatives are mostly feasible.

Chapter 3 handles the case where the demanded derivatives hit the physical limit.

The Figure 8 succeeds because the controller understands the tangent-space boundary and makes the correct trade-off.

34. The cleanest mental model

You can understand Goh's Chapters 2–3 with this hierarchy:

Level 1: vehicle motion variables

$$\phi = \text{where the velocity vector points}$$

$$r = \text{how fast the body yaws}$$

$$\beta = \text{angle between body and velocity vector}$$

$$\dot{\beta} = \dot{\phi} - r$$

Level 2: Chapter 2 controller

Use

$$\dot{\phi}_{\text{des}}$$

to fix path error.

Use

$$\dot{r}_{\text{des}}$$

to make yaw rate track the synthetic yaw rate needed for sideslip stabilization.

Invert the nonlinear model:

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}) \to (\delta, \gamma).$$

Leave speed untracked, but choose the stable branch of the feasible derivative surface so V behaves as a stable zero dynamic.

Level 3: Chapter 2 wheel dynamics

Convert

$$\gamma_{\text{des}} \to \omega_{\text{des}} \to \tau.$$

Do this because rear tire force direction is set by slip velocity, and slip velocity depends on wheel speed.

Level 4: Chapter 3 planning

Plan a Figure 8 transition with nonlinear optimization, actuator slew limits, torque limits, power limits, and endpoint drifting equilibria.

Level 5: Chapter 3 tracking

Recognize that the Figure 8 lives near the tangent-space boundary.

When tracking commands are infeasible:

$$\text{do not minimize path error blindly.}$$

Instead:

$$\text{project onto the feasible boundary while preserving yaw/sideslip dynamics.}$$

This prevents loss of yaw authority and avoids spinout.

35. Why these chapters are foundational for the rest of the thesis

Chapter 2 gives the controller structure.

Chapter 3 reveals the key safety limitation.

The limitation is not merely:

$$\text{open-loop instability.}$$

Goh's deeper point is:

$$\text{The critical danger is losing the ability to recover sideslip/yaw because the tangent space collapses.}$$

This insight directly leads to Chapter 4, where Goh defines a safe operating envelope based not on open-loop stability, but on whether the closed-loop controller can eventually recover sideslip.

So the thesis progression is:

$$\text{Chapter 2: How to drift along a path.}$$

$$\text{Chapter 3: How to survive derivative limits during dynamic drift transitions.}$$

$$\text{Chapter 4: How to define the boundary beyond which recovery is impossible.}$$

That is why Chapters 2 and 3 are the technical core of the dissertation.

Goh_AutomatedVehicleControl-035-0...
PDF

Goh_AutomatedVehicleControl-067-0...
PDF

Double check your assistance explanation materials against Chapter 2 and Chapter 3 here to make sure all the important contents are covered and explained in the most detailed way, both physically and mathematically correct and consistent.

已思考 50s

I double-checked my previous explanation against the Chapter 2 and Chapter 3 PDFs you uploaded. The overall structure was correct, but I would make **three important corrections/clarifications** before treating it as learning material:

1. **Do not treat Chapter 2's projection and Chapter 3's projection as the same.**
   Chapter 2 uses a deliberately **naive projection**: first saturate $\dot{\phi}_{\text{des}}$, then project the pair $(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}})$ along the line $\dot{r} = \dot{r}_{\text{des}}$. Chapter 3 replaces this with a more rigorous **yaw-stability-preserving projection** that respects the coupling between $\dot{\phi}_{\text{des}}$ and $\dot{r}_{\text{des}}$. Goh_AutomatedVehicleControl-035... Goh_AutomatedVehicleControl-067...
2. **My earlier mention of "holding the previous actuator solution if the projection line does not intersect" should be removed.**
   That is **not stated in Chapter 3**. Goh's Chapter 3 explanation is: if the nominal command is outside the tangent-space boundary, project onto the tangent-space boundary along the yaw-control-law line. Goh_AutomatedVehicleControl-067...
3. **The rear thrust-angle definition should be understood physically as the angle of the saturated rear force vector.**
   The consistent interpretation across Chapter 2.3 and Chapter 3 is

   $$\gamma = \operatorname{atan2}(F_{yr}, F_{xr}),$$

   so that

   $$F_{xr} = \mu F_{zr} \cos\gamma, \qquad F_{yr} = \mu F_{zr} \sin\gamma.$$

   This is consistent with Chapter 2.3's slip-velocity geometry and Chapter 3's relation

   $$\gamma = -\operatorname{sgn}(\alpha_r) \arccos\left(\frac{F_{xr}}{\mu F_{zr}}\right).$$

   Goh_AutomatedVehicleControl-035... Goh_AutomatedVehicleControl-067...

Below is the corrected full explanation.

Chapter 2: Controller Design for Drifting Along a General Path

Chapter 2's central problem is:

$$\text{How can an autonomous vehicle drift along a general path, not just stabilize one drift equilibrium?}$$

The chapter explicitly says previous work often stabilized a vehicle around a single drift equilibrium, but this is not enough for complex trajectories. Goh's controller is designed without assuming the vehicle is near an equilibrium point and without assuming that the body heading follows the velocity direction. Instead, he uses the rotation rate of the **velocity vector** to track the path. Goh_AutomatedVehicleControl-035...

The key conceptual swap is:

$$\text{normal driving:} \qquad \delta \to \text{path}, \quad \tau \to \text{speed}, \quad \beta \text{ naturally stable.}$$

$$\text{drifting in Goh:} \qquad \delta, \tau \to \text{path and sideslip}, \quad V \text{ untracked but stable.}$$

This is not just wording. It is the actual control architecture.

1. Vehicle states and path variables

Goh uses a three-state planar single-track model:

$$r = \text{yaw rate},$$

$$V = \text{speed magnitude},$$

$$\beta = \text{sideslip angle}.$$

Then he defines the **course angle**

$$\phi = \text{direction of the CG velocity vector}.$$

This is different from body heading ψ. In drifting, ψ and ϕ are not close. The car body may point sideways, but the vehicle still moves along the path through its velocity vector.

The central kinematic identity is

$$\dot{\phi} = \dot{\beta} + r.$$

Equivalently,

$$\dot{\beta} = \dot{\phi} - r.$$

This one equation explains the whole controller:

- $\dot{\phi}$ rotates the **velocity vector**.
- r rotates the **vehicle body**.
- Their difference changes sideslip.

So Goh uses:

$$\dot{\phi} \quad \text{for path tracking,}$$

and

$$\dot{r} \quad \text{for yaw/sideslip stabilization.}$$

The thesis defines ϕ as the velocity-vector direction and states this kinematic relation directly in Section 2.1.1. Goh_AutomatedVehicleControl-035...

2. Path-tracking kinematics

Let

$$e = \text{lateral error},$$

$$s = \text{distance along the reference path},$$

$$\phi_{\text{ref}} = \text{reference path tangent angle},$$

$$\Delta\phi = \phi - \phi_{\text{ref}}.$$

The lateral error dynamics are

$$\dot{e} = V \sin \Delta\phi.$$

For small course error,

$$\dot{e} \approx V \Delta\phi.$$

Then

$$\ddot{e} \approx V \Delta\dot{\phi} + \dot{V} \Delta\phi.$$

Goh assumes

$$V \Delta\dot{\phi} \gg \dot{V} \Delta\phi,$$

so

$$\ddot{e} \approx V \Delta\dot{\phi}.$$

This is a **small course-error assumption**, not a small sideslip assumption. That distinction is very important. The vehicle may have -40° sideslip, but if its velocity vector is nearly tangent to the path, $\Delta\phi$ can still be small. Goh explicitly says this is reasonable because $\Delta\phi$ is driven small in closed-loop operation. Goh_AutomatedVehicleControl-035...

The course-error dynamics are

$$\Delta\dot{\phi} = \dot{\phi} - \dot{\phi}_{\text{ref}},$$

with

$$\dot{\phi}_{\text{ref}} = \kappa_{\text{ref}} \begin{pmatrix} V \cos \Delta\phi \\ 1 - \kappa_{\text{ref}} e \end{pmatrix}.$$

At zero path error and zero course error,

$$e = 0, \qquad \Delta\phi = 0,$$

so

$$\dot{\phi}_{\text{ref}} = V \kappa_{\text{ref}}.$$

This becomes central for the velocity-zero-dynamics argument.

3. Force-based vehicle model

The single-track model uses front and rear forces:

$$F_{yf}, F_{xf}, F_{yr}, F_{xr}.$$

For the rear-wheel-drive MARTY vehicle,

$$F_{xf} = 0,$$

and Goh assumes only steering and throttle actuation, so

$$F_{xr} \geq 0.$$

The equations are

$$\dot{r} = \frac{a F_{yf} \cos\delta + a F_{xf} \sin\delta - b F_{yr}}{I_z},$$

$$\dot{\beta} = \frac{F_{yf} \cos(\delta - \beta) + F_{xf} \sin(\delta - \beta) + F_{yr} \cos\beta - F_{xr} \sin\beta}{mV} - r,$$

$$\dot{V} = \frac{-F_{yf} \sin(\delta - \beta) + F_{xf} \cos(\delta - \beta) + F_{yr} \sin\beta + F_{xr} \cos\beta}{m}.$$

These equations show why drifting is coupled:

$$F_{xr}$$

does not only affect speed; because of the term

$$-F_{xr} \sin\beta$$

it also affects sideslip dynamics. Similarly, steering changes front lateral force, but because the vehicle is at large β, that front lateral force also affects $\dot{V}$. The equations are given in Section 2.1.1. Goh_AutomatedVehicleControl-035...

4. Tire models

The front tire uses a Fiala brush model. The important point is that the front lateral force is a nonlinear function of slip angle and saturates.

The rear tire is assumed fully sliding during drifting, so

$$F_{xr}^2 + F_{yr}^2 = (\mu F_{zr})^2.$$

This is the friction circle.

Therefore, rear drive torque no longer simply increases total tire force. The rear tire is already saturated, so changing wheel speed changes the **direction** of the rear force vector. That is why γ, the rear thrust angle, is an important control variable.

Chapter 2 controller design

The controller is a cascade:

$$(e, \Delta\phi, \beta, r, V) \to (\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}) \to (\delta, \gamma_{\text{des}}) \to \omega_{\text{des}} \to \tau.$$

Goh's block diagram shows this exact structure: imposed error dynamics, nonlinear model inversion, tire-slip mapping, wheelspeed control loop, and vehicle/trajectory dynamics. It also labels V as "free, but stable." Goh_AutomatedVehicleControl-035...

5. Imposed lateral-error dynamics

Goh imposes stable second-order dynamics on lateral error:

$$\ddot{e}_{\text{des}} = -k_p e - k_d \dot{e}.$$

Using

$$\dot{e} \approx V \Delta\phi,$$

$$\ddot{e} \approx V \Delta\dot{\phi},$$

we get

$$V \Delta\dot{\phi}_{\text{des}} = -k_p e - k_d V \Delta\phi.$$

So

$$\Delta\dot{\phi}_{\text{des}} = -\frac{k_p}{V} e - k_d \Delta\phi.$$

Adding the reference course rate gives

$$\dot{\phi}_{\text{des}} = -\frac{k_p}{V} e - k_d \Delta\phi + \dot{\phi}_{\text{ref}}.$$

This is the desired rotation rate of the velocity vector.

Physical meaning:

- If the vehicle is laterally off the path, rotate the velocity vector back toward the path.
- If the velocity vector is not aligned with the path tangent, correct $\Delta\phi$.
- If tracking is perfect, rotate the velocity vector at the rate required by path curvature.

This is why the controller tracks the path using $\dot{\phi}$, not ψ. Goh derives this in Equations 2.8–2.9. Goh_AutomatedVehicleControl-035...

6. Imposed sideslip dynamics

The sideslip error is

$$e_\beta = \beta - \beta_{\text{ref}}.$$

Goh imposes

$$\dot{e}_\beta = -k_\beta e_\beta.$$

Therefore,

$$\dot{\beta}_{\text{des}} = -k_\beta e_\beta + \dot{\beta}_{\text{ref}}.$$

But

$$\dot{\beta} = \dot{\phi} - r.$$

Since $\dot{\phi}_{\text{des}}$ has already been chosen for path tracking, the required yaw rate to achieve the desired sideslip dynamics is

$$r_{\text{syn}} = \dot{\phi}_{\text{des}} - \dot{\beta}_{\text{des}}.$$

Substitute $\dot{\beta}_{\text{des}}$:

$$r_{\text{syn}} = \dot{\phi}_{\text{des}} + k_\beta e_\beta - \dot{\beta}_{\text{ref}}.$$

This is called a **synthetic yaw rate**. It is not an actuator; it is the yaw rate the vehicle should have so that sideslip evolves correctly.

Then Goh imposes yaw-rate tracking:

$$e_r = r - r_{\text{syn}},$$

$$\dot{e}_r = -k_r e_r.$$

So

$$\dot{r}_{\text{des}} = -k_r(r - r_{\text{syn}}) + \dot{r}_{\text{syn}}.$$

Goh also approximates $\dot{r}_{\text{syn}}$, assuming $\dot{V} \approx 0$ and small $\Delta\phi$, and notes that at zero error,

$$\dot{r}_{\text{des}} = \dot{r}_{\text{ref}}.$$

These are Equations 2.10–2.13. Goh_AutomatedVehicleControl-035...

7. What the desired state derivatives mean

The high-level controller outputs

$$\dot{\phi}_{\text{des}}, \qquad \dot{r}_{\text{des}}.$$

These are not actuator commands. They are desired **instantaneous vehicle-motion derivatives**:

$$\dot{\phi}_{\text{des}} = \text{desired course rate},$$

$$\dot{r}_{\text{des}} = \text{desired yaw acceleration}.$$

The controller first decides what instantaneous motion the vehicle should have. Then it asks:

$$\text{What steering and rear force direction produce those derivatives?}$$

That second step is nonlinear model inversion.

8. Nonlinear model inversion

The force-based model maps

$$(\delta, \gamma) \to (\dot{\phi}, \dot{r}, \dot{V}).$$

The controller wants

$$\dot{\phi} = \dot{\phi}_{\text{des}},$$

$$\dot{r} = \dot{r}_{\text{des}}.$$

So it numerically solves

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}) \mapsto (\delta, \gamma_{\text{des}}).$$

This inversion is nonlinear because of:

- large sideslip,
- large steering,
- trigonometric geometry,
- nonlinear front Fiala tire force,
- rear friction-circle saturation,
- coupling between lateral and longitudinal tire forces.

Goh explicitly says the nonlinear single-track model is numerically inverted to map the desired pair $(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}})$ to steering angle and rear thrust angle. Goh_AutomatedVehicleControl-035...

Because there are two inputs and two desired derivatives, $\dot{V}$ is not chosen independently. It is whatever the selected point on the feasible derivative surface gives.

Velocity zero dynamics

This is the most important mathematical idea in Chapter 2.

The controller regulates:

$$e \to 0,$$

$$\Delta\phi \to 0,$$

$$e_\beta \to 0,$$

$$e_r \to 0.$$

The remaining unregulated state is speed V. So Goh studies V as a **zero dynamic**.

At zero error,

$$\dot{\phi}_{\text{des}} = \dot{\phi}_{\text{ref}}.$$

Using the path geometry,

$$\dot{\phi}_{\text{ref}} = V\kappa_{\text{ref}}\frac{\cos\Delta\phi}{1 - e\kappa_{\text{ref}}}.$$

At zero error,

$$\dot{\phi}_{\text{des}} = V\kappa_{\text{ref}}.$$

Also,

$$\dot{r}_{\text{des}} = \dot{r}_{\text{ref}}.$$

Goh then argues that because $\dot{\phi}_{\text{des}}$ depends linearly on V, the velocity zero dynamic is stable if, along a constant-$\dot{r}$ contour,

$$\frac{\partial \dot{V}}{\partial \dot{\phi}} < 0$$

and

$$\dot{V} \text{ has a zero crossing.}$$

Why?

At zero tracking error,

$$\dot{\phi}_{\text{des}} = V\kappa_{\text{ref}}.$$

If V increases, the controller demands a larger $\dot{\phi}$. If larger $\dot{\phi}$ produces smaller $\dot{V}$, then the vehicle decelerates. That is stabilizing:

$$V \uparrow \Rightarrow \dot{\phi}_{\text{des}} \uparrow \Rightarrow \dot{V} \downarrow \Rightarrow V \downarrow .$$

If V decreases, the demanded $\dot{\phi}$ decreases. With the same negative slope, $\dot{V}$ tends to become positive, so the vehicle accelerates:

$$V \downarrow \Rightarrow \dot{\phi}_{\text{des}} \downarrow \Rightarrow \dot{V} \uparrow \Rightarrow V \uparrow .$$

This is the stable-speed mechanism. Goh states this directly in Section 2.2.3. Goh_AutomatedVehicleControl-035...

9. Feasible derivative surface and separatrix

At a fixed state

$$(r, V, \beta),$$

each input pair

$$(\delta, \gamma)$$

produces one derivative triple

$$(\dot{\phi}, \dot{r}, \dot{V}).$$

Since there are two inputs, the feasible set is a two-dimensional surface in the three-dimensional derivative space.

The nonlinear inversion chooses a point on this surface with desired coordinates

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}).$$

But the surface can fold when projected into the $(\dot{\phi}, \dot{r})$ plane. That means the same desired pair could correspond to multiple actuator solutions with different $\dot{V}$. Goh introduces the **separatrix** to select the correct branch.

The separatrix is defined by the local singularity condition

$$\frac{\partial \dot{\phi}}{\partial \delta}\frac{\partial \dot{r}}{\partial F_{xr}} - \frac{\partial \dot{\phi}}{\partial F_{xr}}\frac{\partial \dot{r}}{\partial \delta} = 0.$$

Goh says that if the separatrix lies below the plane

$$\dot{V} = 0,$$

then constraining inversion to the upper surface should give stable velocity zero dynamics. It also makes the mapping from $(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}})$ to actuator commands one-to-one. Goh_AutomatedVehicleControl-035...

This is an important limitation: Goh is not proving global speed stability everywhere. He numerically checks the condition over a broad operating range and finds it valid except at very high yaw rates and very low speeds. Goh_AutomatedVehicleControl-035...

10. Chapter 2 projection into feasible space

For a general trajectory, the desired pair

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}})$$

may fall outside the feasible actuation space.

In Chapter 2, Goh handles this in a deliberately simple way:

1. Saturate $\dot{\phi}_{\text{des}}$ to feasible values.
2. Use the saturated $\dot{\phi}_{\text{des}}$ to compute $\dot{r}_{\text{des}}$.
3. Project the resulting pair into the feasible space along

$$\dot{r} = \dot{r}_{\text{des}}.$$

This "roughly prioritizes" yaw/sideslip dynamics over lateral-error dynamics. But Chapter 3 later shows this is not rigorous enough for highly dynamic transitions. Goh_AutomatedVehicleControl-035...

Wheelspeed dynamics and control

The high-level controller asks for γ_{des}, the rear thrust angle. But the physical actuator is torque τ. Therefore Goh needs:

$$\gamma_{\text{des}} \to \omega_{\text{des}} \to \tau.$$

Because the rear tire is fully sliding, the rear force vector opposes the slip velocity vector. Goh gives

$$\gamma = \arctan\begin{pmatrix} -(V\sin\beta - br) \\ -V\cos\beta + R\omega \end{pmatrix}.$$

Solving for desired wheel speed gives

$$\omega_{\text{des}} = \frac{1}{R}\left[\begin{matrix} -V\sin\beta - br \\ \tan\gamma_{\text{des}} \end{matrix} + V\cos\beta\right].$$

This is the tire-slip mapping. It says: to obtain a desired rear force direction, command a corresponding wheel speed. Goh_AutomatedVehicleControl-035...

The wheel dynamics are

$$I_\omega\dot{\omega} = \tau - RF_{xr}.$$

So torque changes wheel acceleration, and wheel speed changes rear force direction. Goh uses

$$\tau_{\text{command}} = -k_\omega I_\omega(\omega - \omega_{\text{des},f}) + I_\omega\dot{\omega}_{\text{des},f} + RF_{xr,\text{des}}.$$

The first term is feedback, the second is desired acceleration feedforward, and the third balances the tire force torque. The desired wheel speed is filtered before tracking.

This makes

$$\omega \to \omega_{\text{des},f},$$

so that

$$\gamma \to \gamma_{\text{des}}.$$

Goh's reason for doing this is twofold:

First, controlling thrust angle γ is more robust to variation in μF_z than directly commanding F_{xr}. Holding F_{xr} fixed can create large changes in F_{yr} when μF_z changes, whereas holding γ makes both force components scale proportionally. Second, wheelspin dynamics cause significant rear force-generation delay; near the drift equilibrium, the rear force rise time can be more than five times a front lateral-force rise-time estimate. Goh_AutomatedVehicleControl-035...

That is why Chapter 2.3 is not a minor actuator detail. It is what makes the force-level nonlinear inversion experimentally meaningful.

Chapter 2 experimental result

Chapter 2 validates the controller on a slowly changing drifting path with curvature from $1/7$ to $1/20\ \text{m}^{-1}$, speed from 25 to $45\ \text{km/h}$, and sideslip held near -40°. The chapter reports maximum lateral deviation about $-0.36\ \text{m}$, RMS lateral error $0.18\ \text{m}$, maximum sideslip deviation -6.1°, and RMS sideslip error 2.4°. It also notes that speed stays close to the quasi-equilibrium values, supporting the stable velocity-zero-dynamics argument. Goh_AutomatedVehicleControl-035...

When wheelspeed control is disabled and torque is commanded directly as

$$\tau_{\text{command}} = RF_{xr,\text{des}},$$

the vehicle still drifts, but lateral error, course error, sideslip, and yaw rate become larger and more oscillatory. This experimentally confirms that rear wheelspeed dynamics matter. Goh_AutomatedVehicleControl-035...

Chapter 3: Execution of Highly Dynamic Drifting Trajectories

Chapter 3 extends Chapter 2 from slowly changing drift paths to highly transient drift maneuvers, especially the Figure 8 drift. The chapter says the Figure 8 starts and ends beyond the stability limits, requires high yaw rate and high input slew rate, and pushes against limits of achievable state derivatives.

The key Chapter 3 problem is:

> What should the controller do when path tracking and sideslip stabilization cannot both be achieved?

Goh's answer:

> Prioritize yaw/sideslip dynamics, because loss of yaw authority leads to spinout.

11. Trajectory planning

The Figure 8 consists of:

1. steady-state drift,
2. rapid transition to the opposite drift,
3. steady-state opposite drift,
4. another rapid transition.

The rapid transition is planned as a nonlinear optimization problem.

The extended state is

$$X = \begin{bmatrix} r & V & \beta & E & N & \phi & \delta & F_{xr} \end{bmatrix}^T.$$

The input is

$$U = \begin{bmatrix} \dot{\delta} & \dot{F}_{xr} \end{bmatrix}^T.$$

Including δ and F_{xr} as states lets Goh constrain actuator slew rates directly.

The problem is spatially discretized with step ds, total length

$$s_{\text{total}} = Nds,$$

and dynamics are divided by $V = ds/dt$ and integrated by RK4. Goh_AutomatedVehicleControl-067...

12. Endpoint constraints

The initial and final states are drifting equilibria:

$$\dot{r} = 0, \qquad \dot{V} = 0, \qquad \dot{\beta} = 0.$$

The endpoints are selected by desired sideslip and curvature. For the experimental trajectory,

$$\beta_I = -\beta_F = -0.7\ \text{rad},$$
$$\kappa_I = -\kappa_F = \frac{1}{18}\ \text{m}^{-1},$$
$$D = 43\ \text{m}.$$

A nominal clothoid gives the final pose and initial guess, but the optimized trajectory is constrained to the clothoid only at the start and end, not throughout the transition. Goh_AutomatedVehicleControl-067...

13. Constraints and cost

Goh constrains:

$$\dot{\delta}, \quad \dot{F}_{xr}, \quad \delta, \quad F_{xr}, \quad \tau, \quad P.$$

Because the maneuver requires large wheel slip and rapid force changes, he explicitly includes drivetrain torque and power limits. Using the saturated-rear-tire relation,

$$\gamma = -\operatorname{sgn}(\alpha_r)\arccos\begin{pmatrix} F_{xr} \\ \mu F_{xr} \end{pmatrix},$$

and

$$\omega_k = \frac{1}{R}\left[\begin{matrix} -V\sin\beta - br \\ \tan\gamma \end{matrix} + V\cos\beta\right],$$

he approximates torque and power over each stage. The chosen constraints include $P_{\max} = 250\ \text{kW}$, $\tau_{\max} = 4.4\ \text{kNm}$, $|\dot{\delta}|_{\max} = 2\ \text{rad/s}$, $|\dot{F}_{xr}|_{\max} = 20\ \text{kN/s}$, and $|\delta|_{\max} = 0.66\ \text{rad}$. Goh_AutomatedVehicleControl-067...

The cost is

$$J = \sum_{k=1}^{N-1} U_k^T R U_k + \sum_{k=1}^{N} Q\left(|\beta_k - \bar{\beta}| - \Delta\beta^*\right)^2.$$

The first term penalizes actuator slew rate. The second term encourages rapid sideslip transition by penalizing sideslip values that are not close to either endpoint value. Increasing Q gives faster sideslip transitions, higher peak yaw rates, and faster steering sweeps. Goh chooses $Q = 75$ as a compromise between aggressiveness and actuator headroom. Goh_AutomatedVehicleControl-067...

Tangent space in Chapter 3

At a fixed state

$$(r, V, \beta),$$

the set of all possible derivatives

$$(\dot{r}, \dot{\phi}, \dot{V})$$

from all feasible inputs is called the **tangent space**.

Because there are two main inputs, this tangent space is a two-dimensional manifold in a three-dimensional derivative space. If you choose two derivatives, for example $(\dot{r}, \dot{\phi})$, the third derivative $\dot{V}$ is determined. Goh introduces this explicitly in Section 3.2.1. Goh_AutomatedVehicleControl-067...

The Chapter 2 controller commands

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}).$$

Therefore, the important view is the projection of the tangent space into the

$$\dot{\phi} - \dot{r}$$

plane.

The boundary of this projection tells the controller which combinations of path-course rate and yaw acceleration are physically possible.

14. Why the Figure 8 must operate near limits

Goh overlays the planned Figure 8 trajectory on the tangent-space boundary and shows that the planned maneuver comes close to the physical derivative limits.

A natural question is: why not plan with a big safety margin?

Goh explains that three boundaries correspond to actuator extremes and can be avoided by restricting actuator use. But the fourth boundary is a **fold** in the tangent surface. Near that fold, $\dot{V}$ changes rapidly for small changes in $(\dot{\phi}, \dot{r})$. If you leave a large buffer from this fold in the $\dot{\phi} - \dot{r}$ plane, you eliminate access to negative $\dot{V}$, meaning the vehicle would be restricted to trajectories with increasing speed only. That restriction is too severe for useful dynamic drifting. Goh_AutomatedVehicleControl-067...

So Chapter 3's important insight is:

Highly dynamic drifting fundamentally requires operating near derivative limits.

It is not simply that the controller is weak. The maneuver itself lives near the boundary of what the car can do.

15. Why yaw/sideslip must be prioritized

Goh studies how the tangent-space boundary changes as sideslip increases. At the Figure 8 drift equilibrium,

$$r = 0.66 \text{ rad/s},$$

$$V = 11.8 \text{ m/s},$$

$$\beta = -40^\circ.$$

He then varies β from -30° to -50°. As sideslip magnitude increases above 40°, the feasible derivative set shrinks rapidly. This is caused by steering saturation and loss of accessible front lateral force.

Goh_AutomatedVehicleControl-067...

The maximum achievable front slip angle is written as

$$\alpha_{F,\max} = \arctan\begin{pmatrix} V\sin\beta + ar \\ V\cos\beta \end{pmatrix} + \delta_{\max}.$$

The exact sign of the saturation condition depends on turn direction and sign convention, but the physical condition is: once the front tire reaches the useful saturation boundary under maximum countersteer, the vehicle loses the ability to further alter front lateral force in the needed direction.

At $\beta = -50^\circ$, Goh observes that the vehicle only has access to one sign of yaw acceleration; there is no input combination that can reduce yaw rate in the needed direction. This means the vehicle can no longer recover sideslip, and it will spin out if this continues.

Goh_AutomatedVehicleControl-067...

Therefore:

Path error is recoverable. Loss of yaw/sideslip authority may not be.

That is why Chapter 3 prioritizes yaw/sideslip dynamics.

16. Yaw-stability-preserving projection

When the nominal Chapter 2 controller command

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}})$$

falls outside the feasible tangent-space boundary, both path tracking and sideslip stabilization cannot be satisfied.

Goh formulates the desired trade-off as

$$\min \ \dot{\phi}(x,u) - \dot{\phi}_{\text{des}}(x,p)$$

subject to

$$\dot{r}(x,u) = \dot{r}_{\text{des}}\left(x, p, \dot{\phi}_{\text{des}}(x,p)\right),$$

$$u_{\min} \le u \le u_{\max}.$$

This means:

- preserve yaw/sideslip dynamics,
- sacrifice path-course-rate tracking as little as necessary,
- stay within actuator limits.

Goh then avoids solving this nonlinear optimization online by turning it into geometry. From the yaw control law,

$$\dot{r}_{\text{des}} = -k_r(r - r_{\text{syn}}) + \dot{r}_{\text{syn}},$$

and

$$r_{\text{syn}} = \dot{\phi}_{\text{des}} + k_\beta e_\beta - \dot{\beta}_{\text{ref}},$$

Goh obtains

$$\dot{r}_{\text{des}} = \left[k_r(-r + k_\beta e_\beta - \dot{\beta}_{\text{ref}}) + \dot{r}_{\text{syn}}\right] + k_r\dot{\phi}_{\text{des}}.$$

So in the $\dot{\phi} - \dot{r}$ plane, preserving the yaw/sideslip control law means staying on a line with slope k_r. If the nominal point is outside the tangent space, the controller projects it onto the tangent-space boundary along this line.

Goh_AutomatedVehicleControl-067...

This is **not** the same as Chapter 2's naive projection.

Chapter 2 roughly kept $\dot{r}$ fixed.

Chapter 3 preserves the **relationship** between $\dot{r}_{\text{des}}$ and $\dot{\phi}_{\text{des}}$.

That difference is the technical core of Chapter 3.

17. Boundary computation and null case

To make projection efficient, Goh computes the tangent-space boundary. One part of the boundary corresponds to the fold/separatrix from Chapter 2. By propagating the separatrix condition through the model, he obtains

$$\gamma = \arctan\begin{pmatrix} q\cos\beta + mvb/I_z \\ -q\sin\beta \end{pmatrix},$$

where

$$q = \frac{\partial\dot{r}}{\partial\delta}\frac{\partial\delta}{\partial\dot{\phi}}.$$

This is defined only where the relevant derivatives have the required signs. The remaining boundary segments are constant-input contours.

Goh_AutomatedVehicleControl-067...

To evaluate yaw prioritization experimentally, Goh defines a null case: project along the $\dot{\phi}$ axis. This appears at first to prioritize yaw because it keeps the original $\dot{r}$ demand. But it fails to account for the coupling between $\dot{r}_{\text{des}}$ and $\dot{\phi}_{\text{des}}$, which is why it is not truly yaw-stability preserving.

Goh_AutomatedVehicleControl-067...

Chapter 3 experimental result

Goh conducts two experiments:

1. yaw prioritization active,
2. yaw prioritization disabled / naive projection.

With yaw prioritization active, the vehicle completes two laps of the Figure 8. Lateral error peaks at about 1.3 m near transitions, but after the transitions the error rapidly returns to generally less than 0.4 m. Sideslip overshoot is up to about 5°, yaw rate peaks at 2 rad/s, and steering sweeps through its full $\pm 38^\circ$ range in about one second.

Goh_AutomatedVehicleControl-067...

With the naive projection, the states initially follow the successful case, but shortly after the peak yaw rate they diverge. Sideslip runs away, the tangent space shrinks, the vehicle loses yaw authority, and it spins out.

Goh_AutomatedVehicleControl-067...

This validates the main Chapter 3 claim:

At extreme sideslip, the critical issue is not open-loop instability alone; it is loss of control authority.

Final corrected learning summary

The cleanest way to remember Chapters 2 and 3 is this:

Chapter 2

Goh designs a general-path drifting controller.

The controller controls:

$$e \quad \text{and} \quad \beta.$$

It does this by commanding:

$$\dot{\phi}_{\text{des}} \quad \text{and} \quad \dot{r}_{\text{des}}.$$

Then it uses nonlinear inversion:

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}) \to (\delta, \gamma_{\text{des}}).$$

Because both inputs are already used, speed is not explicitly controlled. But speed can be stable as a zero dynamic if

$$\frac{\partial\dot{V}}{\partial\dot{\phi}} < 0$$

and there is a $\dot{V} = 0$ crossing along constant $\dot{r}$. The separatrix selects the branch of the feasible derivative surface that gives this behavior. Rear wheel dynamics are then handled by mapping

$$\gamma_{\text{des}} \to \omega_{\text{des}} \to \tau.$$

Chapter 3

Goh extends the controller to a highly dynamic Figure 8 drift.

The planned trajectory lives near the tangent-space boundary. Therefore, during tracking, the nominal controller sometimes asks for physically impossible $(\dot{\phi}, \dot{r})$. In that case, Goh prioritizes yaw/sideslip dynamics over path tracking by projecting onto the feasible tangent-space boundary along the yaw-control-law line.

This succeeds because it prevents the vehicle from entering a region where the tangent space collapses and yaw authority is lost.

So the true progression is:

Chapter 2: how to drift along a path.

Chapter 3: how to keep control when the desired drift maneuver hits physical derivative limits.

And the deepest lesson is:

Extreme vehicle control is not only about tracking references; it is about preserving recoverability.

資料來源

Now I am using direct optimal control to design circular, figure-eight and general drifting trajectories of a car. I have a simple two-track vehicle model for trajectory planning and I have a 14DOF high-fidelity vehicle model for simulation. At the moment I do not have any drifting controller yet. According to the Goh's thesis, how would you suggest me to use direct optimal control and my models to design the drifting

trajectories? Moreover, I do not plan to copy Goh's controller for my own project, but to use MPC for my project. What ideas and spirits can you suggested to be borrowed from Goh's thesis so that the trajectories designed by direct optimal control and a potential not-so-

顯示更多

已思考 53s

My strongest recommendation is: **use Goh's thesis less as a controller template and more as a feasibility philosophy for trajectory generation and MPC design**. You do not need to copy his nonlinear inversion controller. But you should borrow three core ideas:

$$\text{Plan and track path} + \text{sideslip/yaw, not path} + \text{speed only.}$$

$$\text{Include actuator, wheel, tire, and rate limits already in the direct OCP.}$$

$$\text{Treat recoverability / yaw authority as a planning and MPC priority.}$$

Goh's Chapter 2 explicitly treats lateral error and sideslip as the control objectives, uses desired state derivatives as the bridge to actuation, leaves speed untracked but stable, and emphasizes saturated rear-tire wheel-speed dynamics. Goh_AutomatedVehicleControl-035... Chapter 3 then shows that highly dynamic Figure-8 drifting must operate near the limits of achievable state derivatives, so when tracking objectives conflict, yaw/sideslip authority should dominate path accuracy. Goh_AutomatedVehicleControl-067...

1. How I would structure your full workflow

You currently have:

$$\text{CasADi/IPOPT direct OCP}$$

$$\text{simple two-track planning model}$$

$$\text{14DOF high-fidelity simulation model}$$

$$\text{future acados MPC in Simulink}$$

I would use them in this hierarchy:

$$\text{Two-track model} = \text{trajectory design and MPC prediction model}$$

$$\text{14DOF model} = \text{truth model / validation model / model-error detector}$$

$$\text{MPC} = \text{closed-loop tracker with feedforward from direct OCP}$$

Do **not** start by putting the full 14DOF model inside direct OCP or MPC. Use the 14DOF model to test whether the planned trajectory is genuinely feasible. Then iteratively tighten the two-track OCP until the 14DOF simulation can follow it.

2. Circular drifting trajectories

For circular drifting, do not first solve a full trajectory OCP. First solve a **drift equilibrium problem**.

Choose a curvature

$$\kappa = \frac{1}{R_{\text{circle}}},$$

a target sideslip

$$\beta_{\text{ref}},$$

and maybe either speed V or let V be optimized.

At a steady circular drift,

$$\dot{r} = 0, \qquad \dot{V} = 0, \qquad \dot{\beta} = 0,$$

and

$$\dot{\phi} = V\kappa, \qquad r = \dot{\phi} - \dot{\beta} = V\kappa.$$

So the equilibrium solve should find:

$$x_{\text{eq}}, u_{\text{eq}} = \{V, r, \beta, \delta, T_i, \omega_i, F_{x,i}, F_{y,i}, F_{z,i}\}$$

such that the vehicle is force/moment balanced.

For a two-track model, I would include at least:

$$\delta, \quad T_{RL}, T_{RR}, \quad \omega_{RL}, \omega_{RR}, \quad F_{z,i}, \quad F_{x,i}, F_{y,i}, \quad \lambda_i, \alpha_i.$$

Even if your direct OCP later uses simplified inputs, your equilibrium search should check tire utilization:

$$\rho_i = \left(\frac{F_{x,i}}{\mu F_{z,i}}\right)^2 + \left(\frac{F_{y,i}}{\mu F_{z,i}}\right)^2.$$

For drifting, rear tires should usually be near saturation:

$$\rho_R \approx 1,$$

but the front tires should not necessarily be forced to full saturation. Leave enough front authority for yaw control.

Then test this equilibrium in the 14DOF model. If the 14DOF vehicle cannot hold it even with feedback, the two-track equilibrium is too optimistic.

3. Figure-eight trajectories

For Figure-8 drifting, follow Goh's planning spirit very closely, but not necessarily his controller.

Goh plans a transition between two drift equilibria using nonlinear optimization. His Chapter 3 extended state includes dynamic states, global pose, steering angle, and rear longitudinal force, while actuator slew rates are used as inputs; this lets him constrain steering-rate and force-rate directly. Goh_AutomatedVehicleControl-067...

For your two-track OCP, I would use a multi-phase structure:

$$\text{Entry} \to \text{left drift equilibrium} \to \text{left-to-right transition} \to \text{right drift equilibrium} \to \text{right-to-left transition} \to \text{exit.}$$

The transition phases are the most important.

Your OCP state should include actuator states, not only vehicle states:

$$x = [X, Y, \psi, V, \beta, r, \delta, \omega_i, T_i]$$

or, in road-relative form,

$$x = [s, n, \Delta\phi, V, \beta, r, \delta, \omega_i, T_i].$$

Then use rate variables as controls:

$$u = [\dot{\delta}, \dot{T}_i]$$

or

$$u = [\dot{\delta}, \dot{F}_{x,i}].$$

This is important because Figure-8 drifting is often limited by **how quickly** steering and drive forces can change, not only by their maximum values. Goh explicitly includes steering-rate, rear-force-rate, torque, and power constraints in the Figure-8 planning problem and uses conservative actuator limits to leave feedback headroom. Goh_AutomatedVehicleControl-067...

Your constraints should include:

$$|\delta| \le \delta_{\max}, \qquad |\dot{\delta}| \le \dot{\delta}_{\max},$$

$$|T_i| \le T_{\max}, \qquad |\dot{T}_i| \le \dot{T}_{\max},$$

$$P_i = T_i\omega_i \le P_{\max},$$

$$F_{z,i} > F_{z,\min},$$

$$\rho_i \le \rho_{\max} < 1$$

for some tires if you want margin, or

$$\rho_R \approx 1$$

if you intentionally want rear saturation.

For a first working version, I would not ask for the fastest possible Figure-8. I would start with a **mild Figure-8**, then gradually increase sideslip, yaw rate, and transition aggressiveness through continuation.

4. General drifting trajectories

For general drifting paths, I would not prescribe speed too strongly. Goh's Chapter 2 shows why: in drifting, steering and throttle are better spent on path and sideslip/yaw behavior, while speed can be a free but stable internal variable under the right coupling. Goh_AutomatedVehicleControl-035...

In direct OCP, this means:

$$\text{hard objective: path and sideslip/yaw feasibility}$$

$$\text{soft objective: speed regularization}$$

A good general-path OCP cost is:

$$J = \int \left[w_n n^2 + w_{\Delta\phi}\Delta\phi^2 + w_\beta(\beta - \beta_{\text{ref}})^2 + w_r(r - r_{\text{ref}})^2 + w_u\|u\|^2 + w_{\dot{u}}\|\dot{u}\|^2\right] dt.$$

But I would keep

$$w_V$$

moderate or small at first. Do not force exact speed tracking unless you already know the speed is feasible. Let the OCP find the speed that makes the drift dynamically possible.

For time-optimal drifting, use

$$J = \int 1\, dt$$

or, in space domain,

$$J = \int \sigma\, ds, \qquad \sigma = \frac{dt}{ds}.$$

But add strong regularization and constraints, otherwise IPOPT may produce a trajectory that is mathematically fast but impossible for the 14DOF model to track.

5. What to borrow from Goh for your MPC

Since you do not want to copy Goh's nonlinear inversion controller, your MPC should replace his inversion/projection logic. But the **priorities** should remain.

Goh's controller says:

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}) \to (\delta, \gamma)$$

through nonlinear inversion.

Your MPC can instead solve:

$$\min_{u_{0:N}} \sum \ell(x_k, u_k)$$

subject to the two-track prediction model and constraints.

So you do not need explicit nonlinear inversion. The MPC will implicitly decide steering and torque. But the MPC cost should reflect the same priorities:

$$\text{sideslip/yaw recovery} > \text{course/path tracking} > \text{speed tracking}.$$

Concretely, in acados, use a time-varying reference from direct OCP:

$$x_{\text{ref}}(t), u_{\text{ref}}(t)$$

but track only selected outputs strongly:

$$y = [n, \Delta\phi, \beta, r, V, \delta, T_i].$$

Use high weights on:

$$\beta - \beta_{\text{ref}}, \qquad r - r_{\text{ref}}, \qquad \Delta\phi,$$

medium weights on:

$$n,$$

and lower weights on:

$$V - V_{\text{ref}}.$$

This is very important. If your MPC over-penalizes path error during a drift transition, it may sacrifice yaw/sideslip recovery and create exactly the failure mode Goh warns about in Chapter 3: the vehicle overshoots sideslip, the feasible derivative set shrinks, and yaw authority collapses. Goh's Figure-8 experiment shows that disabling yaw prioritization made the vehicle spin out, while yaw-prioritized tracking completed the maneuver. Goh_AutomatedVehicleControl-067...

6. Add "Goh-style feasibility margins" to your direct OCP

This is probably the most useful thing for you.

For each point along your planned trajectory, compute a reduced-model approximation of the feasible derivative set:

$$\mathcal{T}(x) = \{(\dot{\phi}, \dot{r}, \dot{V}) : u \in \mathcal{U}\}.$$

This is Goh's tangent-space idea. In Chapter 3, he uses this to show that highly dynamic drifting operates near the limits of achievable state derivatives. Goh_AutomatedVehicleControl-067...

For your project, you do not need to reproduce his exact tangent-space projection. Instead, use the same idea as a **diagnostic and planning constraint**.

At every OCP node, compute:

$$m_{\dot{r}}^{+} = \dot{r}_{\max}(x) - \dot{r}_{\text{ref}},$$
$$m_{\dot{r}}^{-} = \dot{r}_{\text{ref}} - \dot{r}_{\min}(x).$$

Also compute a recovery metric such as:

$$m_{\beta,\text{recover}} = \min_{u \in \mathcal{U}} \text{sgn}(\beta)\dot{\beta}(x, u).$$

For recovery, you want there to exist an input such that:

$$\text{sgn}(\beta)\dot{\beta} < 0,$$

meaning the controller can reduce sideslip magnitude if needed.

If your trajectory has long segments where no input can reduce $|\beta|$, then the direct OCP has planned a trajectory that may be open-loop possible but closed-loop dangerous.

This is the main spirit of Goh Chapter 3:

> Do not only ask whether the nominal trajectory is feasible. Ask whether nearby errors are recoverable.

7. Make the two-track OCP conservative relative to the 14DOF model

Your two-track model will almost certainly be optimistic unless you tighten it.

I would use conservative margins:

$$\mu_{\text{OCP}} = 0.85\mu_{\text{14DOF}},$$
$$\delta_{\max,\text{OCP}} = 0.85\delta_{\max},$$
$$\dot{\delta}_{\max,\text{OCP}} = 0.75\dot{\delta}_{\max},$$
$$T_{\max,\text{OCP}} = 0.8T_{\max},$$
$$P_{\max,\text{OCP}} = 0.8P_{\max}.$$

These numbers are examples, not universal constants. The principle is what matters: **do not use all actuator authority in the open-loop trajectory**, because the MPC needs leftover authority to correct the 14DOF plant.

Goh does this in spirit by using conservative torque and power constraints to leave headroom for closed-loop feedback in the Figure-8 planning problem. Goh_AutomatedVehicleControl-067...

8. Include wheel-speed or slip dynamics in the OCP/MPC

This is another major lesson from Goh.

Goh argues that rear wheelspeed dynamics matter during drifting because saturated rear-tire force direction depends on slip velocity, and wheel speed affects that slip velocity. His Chapter 2 therefore uses a cascaded wheel-speed loop rather than assuming torque instantly gives rear force. Goh_AutomatedVehicleControl-035...

For your system, I would not let the OCP directly command ideal tire forces unless you also constrain their rate and map them to physically achievable wheel speeds/torques.

Better options:

Option A: include wheel speeds in the two-track model

Use

$$I_{\omega,i}\dot{\omega}_i = T_i - RF_{x,i}.$$

Then tire forces come from slip ratio and slip angle.

This is the most physically consistent option.

Option B: command tire forces but add actuator dynamics

Use

$$\dot{F}_{x,i} = \frac{1}{\tau_F}(F_{x,i,\text{cmd}} - F_{x,i})$$

and constrain

$$|\dot{F}_{x,i}| \le \dot{F}_{x,\max}.$$

This is simpler and may be better for acados.

Option C: use OCP force inputs, but validate torque/wheel-speed afterward

This is the easiest, but riskiest. It is acceptable for early trajectory design, but not for final feasibility claims.

For your final pipeline, I would use Option A in direct OCP if IPOPT can handle it, and Option B in MPC if acados needs a simpler model.

9. Suggested direct OCP formulation

For circular, Figure-8, and general drifting, a practical OCP is:

$$\min \int_0^T \left[w_n n^2 + w_{\Delta\phi}\Delta\phi^2 + w_\beta(\beta - \beta_{\text{ref}})^2 + w_r(r - r_{\text{ref}})^2 + w_V(V - V_{\text{ref}})^2 + w_u\|u - u_{\text{ff}}\|^2 + w_{\dot{u}}\|\dot{u}\|^2 + w_\rho \sum_i \text{softplus}(\rho_i - \rho_{\max})^2 \right] dt$$

subject to:

$$\dot{x} = f_{2\text{track}}(x, u),$$
$$|\delta| \le \delta_{\max}, \quad |\dot{\delta}| \le \dot{\delta}_{\max},$$
$$|T_i| \le T_{\max}, \quad |\dot{T}_i| \le \dot{T}_{\max},$$
$$T_i\omega_i \le P_{\max},$$
$$F_{z,i} \ge F_{z,\min},$$
$$\rho_i \le \rho_{\max} + \epsilon_i, \qquad \epsilon_i \ge 0.$$

For drifting, do not make all friction constraints too strict. Rear tires may intentionally be near saturation. Instead, distinguish:

$$\rho_{\text{rear}} \approx 1$$

for drift generation, but preserve front authority:

$$\rho_{\text{front}} \le \rho_{\text{front,max}} < 1$$

or include a yaw-authority margin constraint.

10. Suggested acados MPC formulation

For the MPC, I would use the direct OCP result as a feedforward reference:

$$x_{\text{ref},k}, \qquad u_{\text{ref},k}.$$

Then solve for corrections:

$$u_k = u_{\text{ref},k} + \Delta u_k.$$

The prediction model should be your two-track model, possibly augmented with first-order actuator/wheel-force dynamics.

The 14DOF model should be the Simulink plant.

Use output tracking:

$$y_k = [n, \Delta\phi, \beta, r, V, \delta, T_i].$$

Recommended relative priorities:

$$w_\beta,\ w_r \quad \text{very high},$$

$$w_{\Delta\phi} \quad \text{high},$$

$$w_n \quad \text{medium-high, but soften during transitions},$$

$$w_V \quad \text{low to medium},$$

$$w_{\Delta u} \quad \text{high enough to avoid aggressive correction.}$$

Use soft constraints for path error:

$$n_{\min} - \epsilon_n \le n \le n_{\max} + \epsilon_n,$$

but harder constraints for actuator and recovery-related quantities.

The MPC should not try to exactly track every 14DOF state. Track the low-dimensional drift-relevant outputs and let the 14DOF suspension/wheel-load dynamics respond naturally, while constraining dangerous quantities such as normal load loss, tire saturation, and actuator limits.

11. Validation loop with the 14DOF model

For every OCP trajectory, run this validation sequence:

1. **Open-loop replay on 14DOF**
   Apply the OCP steering/torque commands directly.
2. **Check mismatch**
   Compare:
   $$n, \Delta\phi, \beta, r, V, F_{z,i}, \rho_i, \omega_i, T_i, P_i.$$
3. **Closed-loop MPC replay**
   Use acados MPC with OCP feedforward.
4. **Compute tracking and authority margins**
   Especially check whether the 14DOF trajectory ever enters a state where sideslip cannot be reduced.
5. **Tighten the OCP**
   If the 14DOF plant needs too much correction, reduce aggressiveness, reduce friction assumptions, reduce actuator limits, or add yaw-authority margin.

This is the correct way to make the simple planning model and 14DOF model work together.

12. Practical development order

I would develop your project in this order:

Step 1: circular drift equilibrium library

Generate a table over:

$$\kappa, \beta, V$$

and solve for:

$$\delta, T_i, \omega_i, F_{x,i}, F_{y,i}, F_{z,i}.$$

Validate each point in the 14DOF model.

Step 2: quasi-equilibrium circular path

Track one circular drift with MPC.

Do not start with Figure-8.

Step 3: slow curvature-changing drift

Generate a slowly changing path like Goh Chapter 2. This tests whether your OCP + MPC can handle general drifting without extreme transitions.

Step 4: mild Figure-8

Use low $|\beta|$, low yaw rate, slow transition.

Step 5: aggressive Figure-8

Gradually increase:

$$|\beta|, \quad |\dot\beta|, \quad |r|, \quad |\dot r|, \quad |\dot\delta|.$$

Step 6: general drifting path

Only after circular and Figure-8 work, attempt arbitrary drifting trajectories.

This continuation strategy is much more likely to succeed than directly optimizing an aggressive general drift.

13. The most important design rule

Do not design trajectories that are merely open-loop feasible for the two-track model.

Design trajectories that are:

$$\text{nominally feasible}$$

$$\text{trackable with actuator headroom}$$

$$\text{recoverable after sideslip/yaw error}$$

That is the main lesson to borrow from Goh.

In Chapter 3, the failed case is not simply "bad path tracking." It fails because sideslip overshoots, the tangent space shrinks, and the vehicle loses yaw authority. Goh_AutomatedVehicleControl-067… For your project, the MPC should be designed so that, when path tracking and sideslip recovery conflict, it sacrifices path error first and preserves yaw/sideslip recovery.

So your project should not copy Goh's controller, but it should copy this hierarchy:

$$\text{recoverability first}$$

$$\text{sideslip/yaw tracking second}$$

$$\text{path tracking third}$$

$$\text{speed tracking last or soft}$$

That is the best way to make direct-OCP drifting trajectories feasible for a 14DOF high-fidelity vehicle model and a not-too-complicated MPC.

I think you may give me your previous answers a bit too fast without thinking very carefully. Please refer to Goh's thesis especially Chapter 2 and 3 and then give me suggestions on my own project, so that from my trajectory design using direct optimal control, to MPC, and then to MPC controlling the 14DOF vehicle model are all theoretically sound and practically doable and at last the whole design, control and simulation chain can mostly support the successful drifting for my 14DOF model.

已思考 48s ›

You are right to be cautious. A useful answer for your project should not be "copy Goh with MPC instead of nonlinear inversion." That would be too shallow. The better interpretation is:

> Use Goh's thesis as a design philosophy for feasibility, trackability, and recoverability.

Your toolchain should become:

$$\text{direct OCP} \rightarrow \text{trackable drifting reference} \rightarrow \text{MPC with correct priorities} \rightarrow \text{14DOF validation and tightening.}$$

Goh's Chapter 2 gives the key idea that, in drifting, the controlled objectives should be **path and sideslip**, not path and speed; speed may be soft/untracked because the coupled dynamics can stabilize it. Chapter 3 gives the key idea that highly dynamic drifting is limited by the **set of achievable state derivatives**, and that when path tracking conflicts with yaw/sideslip recovery, yaw/sideslip must win. Goh_AutomatedVehicleControl-067… Goh_AutomatedVehicleControl-035…

1. The main correction to your design philosophy

For normal driving, a common design chain is:

$$\text{generate path} + \text{speed profile} \rightarrow \text{track path and speed.}$$

For drifting, that is not the best starting point. A better chain is:

$$\text{generate path} + \text{sideslip/yaw behavior} + \text{feasible actuator history}$$

then

$$\text{track path, sideslip, yaw rate, and actuator feedforward, with speed softer.}$$

Goh's Chapter 2 explicitly chooses lateral error e and sideslip β as the control objectives. The controller creates desired state derivatives $\dot\phi_{\text{des}}$ and $\dot r_{\text{des}}$, then maps them to steering and rear thrust angle; speed V is not directly regulated because the two inputs are already used for path and sideslip control. Goh_AutomatedVehicleControl-035…

For your project, this means your direct OCP should not be built around "follow this geometric path at this exact speed." It should be built around:

$$n \approx 0, \qquad \Delta\phi \approx 0, \qquad \beta \approx \beta_{\text{drift}}, \qquad r \approx r_{\text{needed}},$$

with V either optimized or softly regularized.

2. What your planning model should contain

You said your planning model is a simple two-track model. That is fine, but for drifting it must contain the same **physical phenomena** that Goh says are essential:

1. large-angle vehicle kinematics, no small-β or small-δ approximations;
2. nonlinear combined-slip tire behavior;
3. rear tire saturation / high slip;
4. actuator limits and actuator rate limits;
5. wheel-speed or at least longitudinal-force generation dynamics;
6. enough load transfer / normal load modeling that the 14DOF model is not much more pessimistic than the planner.

Goh's Chapter 2 says a simple model can work beyond the stability limits only if physically motivated modifications are included: no small-angle assumptions and explicit consideration of rear wheelspeed

dynamics because rear longitudinal/lateral coupling is a critical input. Goh_AutomatedVehicleControl-035...

So I would not use a pure kinematic, force-input, or idealized two-track model for final trajectory design. A practical planning state could be:

$$x_{\mathrm{OCP}} = [X, Y, \psi, v_x, v_y, r, \delta, \omega_{FL}, \omega_{FR}, \omega_{RL}, \omega_{RR}, T_{FL}, T_{FR}, T_{RL}, T_{RR}],$$

or, for road/path-relative planning,

$$x_{\mathrm{OCP}} = [s, n, \Delta\phi, V, \beta, r, \delta, \omega_i, T_i].$$

For rear-wheel drive only, you can simplify the driven-wheel torque states to rear wheels. For a first version, if you do not want all four wheel speeds, include at least the driven rear wheel speeds.

Use actuator-rate controls:

$$u_{\mathrm{OCP}} = [\dot{\delta}, \dot{T}_i],$$

or, if you use force-input planning,

$$u_{\mathrm{OCP}} = [\dot{\delta}, \dot{F}_{x,i}].$$

This is directly in the spirit of Goh Chapter 3: he includes steering angle and rear longitudinal force in the extended state and uses their slew rates as inputs so that rate constraints appear naturally in the nonlinear program. Goh_AutomatedVehicleControl-067...

3. Circular drift design: start with equilibrium maps, not full trajectories

Before circular OCP, solve a family of drift trims.

For each desired curvature

$$\kappa = \frac{1}{R},$$

and desired sideslip

$$\beta_{\mathrm{eq}},$$

solve for a steady drift equilibrium:

$$\dot{V} = 0, \qquad \dot{\beta} = 0, \qquad \dot{r} = 0,$$

with

$$\dot{\phi} = V\kappa, \qquad r = \dot{\phi} - \dot{\beta} = V\kappa.$$

Your trim unknowns may be:

$$V,\ r,\ \delta,\ T_i,\ \omega_i,\ F_{x,i}, F_{y,i}, F_{z,i}.$$

Then enforce tire and actuator feasibility:

$$|T_i| \le T_{\max}, \qquad P_i = T_i\omega_i \le P_{\max},$$

$$|\delta| \le \delta_{\max},$$

$$F_{z,i} > F_{z,\min},$$

$$\rho_i = \left(\frac{F_{x,i}}{\mu F_{z,i}}\right)^2 + \left(\frac{F_{y,i}}{\mu F_{z,i}}\right)^2 \le 1.$$

For drifting, rear ρ_i may intentionally be near one. But do **not** consume all front tire authority. You want front lateral-force authority left for recovery. This is one of the most important practical changes compared with simply finding an open-loop drift equilibrium.

So the first deliverable should be a **trim library**:

$$(\kappa, \beta) \mapsto (V, r, \delta, T_i, \omega_i, F_i).$$

Then replay those trim points in the 14DOF model. If a trim point cannot be held by the 14DOF model even with mild feedback, remove it or tighten the planning model.

4. Figure-eight design: use multi-phase direct OCP

For figure-eight drifting, do not optimize the entire maneuver as one unconstrained black box first. Use phases:

$$\text{entry} \to \text{left drift equilibrium} \to \text{left-to-right transition} \to \text{right drift equilibrium} \to \text{right-to-left transition} \to \text{exit}.$$

This follows Goh's Chapter 3 structure: the circular parts are treated as drift equilibrium segments, and the difficult part is the transition between opposite drift equilibria. Goh formulates that transition as a nonlinear optimization problem constrained by dynamics, actuator range, actuator slew rate, torque, and power. Goh_AutomatedVehicleControl-067...

For your transition OCP, use endpoint constraints like:

$$x(0) = x_{\mathrm{eq,L}}, \qquad x(T) = x_{\mathrm{eq,R}},$$

where both endpoint states are valid trims from your equilibrium library.

The transition cost should not only minimize path error. It should shape the transition:

$$J = \int \left[w_\delta \dot{\delta}^2 + w_{\dot{T}} \dot{T}^2 + w_\beta \ell_\beta(\beta) + w_{\mathrm{track}}\ell_{\mathrm{track}} + w_{\mathrm{margin}}\ell_{\mathrm{margin}} \right] dt.$$

Goh used a sideslip-transition cost that encourages β to rapidly move from one endpoint sideslip to the other while penalizing actuator slew. Increasing that transition weight made sideslip change faster, but also increased peak yaw rate and steering sweep, so he chose a compromise value to preserve actuator headroom. Goh_AutomatedVehicleControl-067...

For your project, this means: start with a **mild transition cost**, not a minimum-time or maximum-aggression figure-eight. Then use continuation:

$$|\beta| = 15^\circ \to 25^\circ \to 35^\circ \to 40^\circ,$$

and

$$\text{slow transition} \to \text{faster transition}.$$

That continuation strategy is much more likely to give a trajectory the 14DOF model can actually follow.

5. General drifting trajectories: do not over-prescribe speed

For general drifting, I would formulate your direct OCP in road/path coordinates:

$$s,\ n,\ \Delta\phi,\ V,\ \beta,\ r.$$

Use:

$$\dot{n} = V \sin\Delta\phi,$$

$$\dot{s} = \frac{V\cos\Delta\phi}{1 - \kappa(s)n},$$

$$\dot{\Delta\phi} = \dot{\phi} - \kappa(s)\dot{s},$$

$$\dot{\beta} = \dot{\phi} - r.$$

This is aligned with Goh's use of course angle ϕ, course error $\Delta\phi$, and lateral error e. He uses the velocity-vector rotation rate $\dot{\phi}$, not vehicle heading alone, to track the path. Goh_AutomatedVehicleControl-035...

Your general OCP cost should look more like:

$$J = \int \left[w_n n^2 + w_{\Delta\phi}\Delta\phi^2 + w_\beta(\beta - \beta_{\mathrm{ref}})^2 + w_r(r - r_{\mathrm{ref}})^2 + w_\delta(\delta - \delta_{\mathrm{ff}})^2 + w_T(T - T_{\mathrm{ff}})^2 + w_u\|\dot{u}\|^2 + w_V(V - V_{\mathrm{soft}})^2 \right] dt.$$

Use w_V modestly. Do not force exact speed unless you already know it is feasible. Goh's Chapter 2 argues that, in drifting, path and sideslip consume the two main input directions, while velocity can be untracked but stable because of the coupling between longitudinal and lateral dynamics. Goh_AutomatedVehicleControl-035...

So in your OCP:

$$\beta,\ r,\ n,\ \Delta\phi \text{ are primary.}$$

$$V \text{ is secondary / soft / optimized.}$$

This is especially important for figure-eight transitions. If you force an exact speed profile, IPOPT may find a trajectory that looks mathematically feasible for the two-track model but leaves no control authority for the MPC.

6. The most important addition: trackability and recoverability checks

This is where I would be more careful than in my previous answer.

Goh Chapter 3 does **not** say "just add actuator constraints and use MPC." It says highly dynamic drifting is hard because the desired trajectory approaches the boundary of the **tangent space**, the set of achievable state derivatives. When the nominal command is outside this set, path tracking and sideslip stabilization cannot both be achieved, so yaw/sideslip must be prioritized. Goh_AutomatedVehicleControl-067...

For your project, after direct OCP, every candidate trajectory should pass a **trackability audit**.

At each OCP node x_k, define the feasible derivative set of your two-track model:

$$\mathcal{T}(x_k) = \left\{ (\dot{\phi}, \dot{r}, \dot{V}) : u \in \mathcal{U}(x_k) \right\}.$$

Here $\mathcal{U}(x_k)$ includes steering limits, torque limits, power limits, wheel-speed constraints, and actuator-rate limits.

Then check where your reference derivative lies:

$$y_k^{\mathrm{ref}} = (\dot{\phi}_k^{\mathrm{ref}}, \dot{r}_k^{\mathrm{ref}}).$$

You want:

$$(\dot{\phi}_k^{\mathrm{ref}}, \dot{r}_k^{\mathrm{ref}}) \in \mathcal{T}_{\dot{\phi}r}(x_k),$$

where $\mathcal{T}_{\dot{\phi}r}$ is the projection of the tangent space into the $(\dot{\phi}, \dot{r})$ plane.

But be careful: Goh also explains that for highly dynamic trajectories, trying to leave a large buffer from every tangent-space boundary may be too restrictive, because one boundary is a fold and avoiding it can remove access to negative $\dot{V}$. So the right rule is not "always stay far from every boundary." The right rule is:

$$\text{Leave margin to actuator-saturation boundaries, but understand and monitor the fold boundary.}$$

For your practical implementation, I suggest three margins.

Margin A: actuator headroom

At every OCP node:

$$|\delta_k| \le 0.8\delta_{\max},$$

$$|\dot{\delta}_k| \le 0.7\dot{\delta}_{\max},$$

$$|T_k| \le 0.8T_{\max},$$

$$P_k \le 0.8P_{\max}.$$

These numbers are not universal. Tune them. The principle is: the OCP must not use all authority, because the MPC needs correction authority.

Goh does this in Chapter 3 by using conservative torque and power limits to leave headroom for feedback. Goh_AutomatedVehicleControl-067...

Margin B: front yaw-authority margin

At high sideslip, Goh shows the tangent space shrinks because steering saturation prevents the front tire from generating the needed yaw authority. In his analysis, as $|\beta|$ grows from about 40° toward 50°, the achievable derivative set rapidly collapses. Goh_AutomatedVehicleControl-067...

For your two-track model, compute a simple recovery margin:

$$m_{\text{rec}}(x) = \min_{u \in \mathcal{U}} \operatorname{sgn}(\beta)\dot{\beta}(x,u).$$

If

$$m_{\text{rec}}(x) < 0,$$

then at least one input can reduce sideslip magnitude.

If

$$m_{\text{rec}}(x) > 0,$$

then every feasible input increases sideslip magnitude. That is dangerous.

Add a soft constraint or penalty:

$$m_{\text{rec}}(x) \le -\epsilon.$$

This is not exactly Goh's Chapter 4 MPRE, but it borrows the Chapter 3 spirit: do not merely ask whether the nominal trajectory is feasible; ask whether the vehicle can recover from errors.

Margin C: MPC correction margin

Simulate small perturbations around the OCP trajectory:

$$\beta + \Delta\beta, \quad r + \Delta r, \quad n + \Delta n.$$

Then check whether the MPC can recover on the two-track prediction model before testing the 14DOF model. This avoids wasting time debugging the 14DOF simulation when the reduced closed-loop problem itself is not robust.

7. What MPC should borrow from Goh, without copying his controller

You do not need to implement Goh's nonlinear inversion. MPC replaces that.

Goh:

$$(\dot{\phi}_{\text{des}}, \dot{r}_{\text{des}}) \to (\delta, \gamma)$$

by model inversion.

Your MPC:

$$\min_{u_{0:N}} J_{\text{MPC}} \quad \text{s.t. two-track dynamics and constraints.}$$

The MPC will implicitly choose steering and torques. But the **objective hierarchy** should be Goh-like.

For the MPC output vector, use something like:

$$y = [n, \Delta\phi, \beta, r, V, \delta, T_i, \omega_i].$$

Use the direct OCP solution as feedforward:

$$x_{\text{ref},k}, u_{\text{ref},k}.$$

Then minimize deviations:

$$J_{\text{MPC}} = \sum_k \left[\|n_k - n_{\text{ref},k}\|^2_{Q_n} + \|\Delta\phi_k - \Delta\phi_{\text{ref},k}\|^2_{Q_\phi} + \|\beta_k - \beta_{\text{ref},k}\|^2_{Q_\beta} + \|r_k - r_{\text{ref},k}\|^2_{Q_r} + \|V_k - V_{\text{ref},k}\|^2_{Q_V} + \|u_k - u_{\text{ref},k}\|^2_R + \|\Delta u_k\|^2_{R_\Delta} \right].$$

But choose weights with this hierarchy:

$$Q_\beta,\ Q_r \quad \text{highest},$$

$$Q_{\Delta\phi} \quad \text{high},$$

$$Q_n \quad \text{medium to high},$$

$$Q_V \quad \text{low to medium}.$$

Why not make Q_n highest? Because in a drift transition, exact path tracking may conflict with yaw/sideslip recovery. Goh's Chapter 3 shows that when both cannot be achieved, prioritizing yaw/sideslip is what prevents spinout. Goh_AutomatedVehicleControl-067...

A very practical MPC trick is to use **adaptive weights**:

If the recoverability margin is healthy,

$$m_{\text{rec}} < -\epsilon_{\text{safe}},$$

use normal path-tracking weights.

If the vehicle approaches loss of yaw authority,

$$m_{\text{rec}} \to 0,$$

then reduce Q_n and increase Q_β, Q_r.

This gives an MPC version of Goh's yaw-prioritizing projection, without copying his controller.

8. MPC model choice: do not use full 14DOF inside MPC first

For a not-too-complicated MPC, I would not start with the full 14DOF model as the acados prediction model.

Use:

$$\text{two-track model in acados MPC}$$

and

$$\text{14DOF model as Simulink plant}.$$

The 14DOF model is your truth model, not your first MPC prediction model.

The MPC prediction model should include enough physics to capture drift, but remain simple enough to solve reliably:

$$x_{\text{MPC}} = [s, n, \Delta\phi, V, \beta, r, \delta, \omega_{RL}, \omega_{RR}]$$

for rear-wheel drive, or four wheel speeds if needed.

Controls:

$$u_{\text{MPC}} = [\dot{\delta}, T_{RL}, T_{RR}]$$

or

$$[\dot{\delta}, \dot{T}_{RL}, \dot{T}_{RR}]$$

if you want torque-rate states.

A good compromise is:

$$\dot{\delta} = u_\delta,$$

$$I_w\dot{\omega}_i = T_i - RF_{x,i},$$

with direct torque commands T_i limited by magnitude, rate, and power.

This is directly motivated by Goh's Chapter 2 conclusion that rear wheelspeed dynamics matter during drifting and that ignoring them creates force phase lag. His experiments without wheelspeed control showed larger oscillations and force loops relative to commanded force. Goh_AutomatedVehicleControl-035...

9. How to connect MPC to the 14DOF model

Your Simulink 14DOF plant should receive the same physical commands used in the MPC:

$$\delta_{\text{cmd}}, \qquad T_{\text{i,cmd}}$$

or brake/drive torques.

Do not have the MPC command ideal tire forces to the 14DOF plant. That creates an artificial interface. If the OCP uses tire forces, convert them to physically feasible torque/wheel-speed references before simulation.

At every simulation step, compute reduced outputs from the 14DOF state:

$$V = \quad v_x^2 + v_y^2,$$

$$\beta = \operatorname{atan2}(v_y, v_x),$$

$$r = \text{yaw rate},$$

$$\phi = \psi + \beta,$$

$$\Delta\phi = \phi - \phi_{\text{ref}}(s),$$

$$n = \text{lateral path error}.$$

Also monitor 14DOF-only quantities:

$$F_{z,i}, \quad \rho_i, \quad \text{suspension travel}, \quad \text{wheel-road contact state}, \quad \text{roll/pitch}, \quad \omega_i, \quad T_i\omega_i.$$

If the 14DOF model fails while the two-track model succeeds, identify the cause:

- If $F_{z,i}$ differs strongly, improve or tighten load transfer in the two-track model.
- If wheel speeds lag, add wheel-speed dynamics or stronger torque-rate limits.
- If front tires saturate earlier, reduce μ, front cornering stiffness, or add front-authority constraints.
- If roll/pitch/suspension causes contact loss, constrain normal load and vertical load variation in planning.
- If the MPC uses all actuator authority, tighten OCP actuator limits and reduce trajectory aggressiveness.

10. The full recommended development sequence

I would use this sequence.

Stage 1: two-track trim validation

Build the drift-equilibrium map:

$$(\kappa, \beta) \to (V, r, \delta, T_i, \omega_i).$$

Validate each point in the 14DOF model.

Deliverable: a set of circular drift equilibria that the 14DOF model can actually hold.

Stage 2: circular drift MPC

Track one circular drift using acados MPC and the 14DOF Simulink plant.

Do not start with figure-eight. If circular drift cannot be stabilized, figure-eight will not work.

MPC priorities:

$$\beta, r, \Delta\phi \quad \text{before} \quad n, V.$$

Stage 3: slowly changing drift path

Generate a quasi-equilibrium path by connecting trim points with slowly changing curvature and sideslip. This mirrors the Chapter 2 experimental path, which was slowly changing and validated the path+sideslip controller before the highly dynamic case. Goh_AutomatedVehicleControl-035...

This stage tests whether your OCP, MPC, and 14DOF model agree over a broad drift range.

Stage 4: mild figure-eight transition

Use multi-phase OCP:

$$\text{left drift equilibrium} \to \text{transition} \to \text{right drift equilibrium}.$$

Use low sideslip first:

$$|\beta| = 15^\circ \text{ to } 25^\circ.$$

Then increase.

Stage 5: tangent-space / recoverability audit

For every candidate figure-eight trajectory, compute:

$$\mathcal{T}_{\phi r}(x_k)$$

and check whether the planned $(\dot{\phi}, \dot{r})$ lies inside it. Also compute the recovery margin:

$$m_{\text{rec}}(x_k) = \min_{u \in \mathcal{U}} \operatorname{sgn}(\beta_k)\dot{\beta}(x_k, u).$$

Do not proceed to 14DOF simulation if the reduced model says the trajectory is unrecoverable.

Stage 6: aggressive figure-eight

Gradually increase:

$$|\beta|, \quad |r|, \quad |\dot{r}|, \quad |\dot{\delta}|, \quad \text{transition speed}.$$

Do not jump directly to Goh-level aggressiveness. Goh's successful figure-eight had yaw rate around $2\,\text{rad/s}$ and a steering sweep of about 75° in less than a second, but this was after a very specific platform, planning method, and controller design. Goh_AutomatedVehicleControl-067...

Stage 7: general drifting trajectory

Only after circular drift, slow drift, and mild figure-eight work should you solve arbitrary drifting trajectories.

11. The direct OCP should produce more than a state reference

Your OCP output should include:

$$x_{\text{ref}}(t), \quad u_{\text{ff}}(t), \quad \dot{x}_{\text{ref}}(t), \quad \text{tire forces}, \quad \text{tire utilization}, \quad \text{normal loads}, \quad \text{recoverability margin}.$$

The MPC should not receive only X, Y, ψ, V. It should receive the drift-relevant reference:

$$n_{\text{ref}}, \quad \Delta\phi_{\text{ref}}, \quad \beta_{\text{ref}}, \quad r_{\text{ref}}, \quad V_{\text{ref}}, \quad \delta_{\text{ff}}, \quad T_{\text{ff}}, \quad \omega_{\text{ff}}.$$

This makes the MPC a **trajectory stabilizer around a dynamically meaningful drift motion**, not just a path follower.

12. Theoretical soundness of the whole chain

Your chain is theoretically sound if each layer answers a different question:

Direct OCP

Question:

$$\text{Does there exist a physically feasible nominal drift trajectory?}$$

Therefore, the OCP must include vehicle dynamics, tire limits, actuator limits, actuator rates, wheel/force dynamics, and at least approximate normal-load behavior.

Trackability audit

Question:

$$\text{Does the nominal trajectory leave enough local authority for feedback?}$$

Therefore, compute tangent-space/recoverability margins.

MPC

Question:

$$\text{Can the reduced model correct tracking errors while preserving yaw/sideslip authority?}$$

Therefore, MPC should prioritize $\beta, r, \Delta\phi$ over exact n, V when the vehicle is near the limit.

14DOF simulation

Question:

$$\text{Do the neglected high-fidelity effects destroy feasibility?}$$

Therefore, monitor load transfer, contact, wheel speeds, suspension motion, tire utilization, and actuator usage.

If a trajectory fails in the 14DOF model, do not immediately tune the MPC harder. First ask which OCP assumption was too optimistic.

13. The most important practical warnings

First, do not use exact speed tracking as a hard objective during drift. Speed should usually be optimized or softly tracked. This is directly aligned with Goh's Chapter 2 velocity-zero-dynamics insight. Goh_AutomatedVehicleControl-035...

Second, do not let the OCP use 100% of tire, steering, torque, or power capacity. The MPC needs headroom.

Third, do not ignore wheel speeds. If your OCP/MPC commands tire force directly, at least add force-rate dynamics and check torque/power feasibility afterward. Better: include wheel-speed dynamics.

Fourth, do not make the MPC track all outputs with equal weights. Drifting needs hierarchy:

$$\text{recoverability} > \text{sideslip/yaw} > \text{course/path} > \text{speed}.$$

Fifth, do not judge a trajectory only by whether the open-loop OCP solution satisfies constraints. Judge it by whether the MPC can recover from perturbations around it.

14. My final recommended architecture

For your current toolchain, I would implement:

$$\text{CasADi/IPOPT two-track direct OCP with wheel/actuator dynamics}$$

producing

$$x_{\text{ref}}, u_{\text{ff}}, \dot{x}_{\text{ref}}, \text{margins}.$$

Then:

$$\text{offline tangent-space and recoverability audit}$$

Then:

$$\text{acados NMPC using the two-track model}$$

tracking

$$n, \Delta\phi, \beta, r, V$$

with priority on

$$\beta, r, \Delta\phi$$

and soft/secondary speed tracking.

Then:

$$\text{Simulink 14DOF plant validation}$$

with iterative tightening of the OCP model until the 14DOF model succeeds.

This is faithful to Goh's thesis without copying Goh's controller. You are replacing his nonlinear inversion and yaw-prioritizing projection with MPC, but you are preserving the real lesson:

$$\text{A drifting trajectory must be feasible, trackable, and recoverable — not merely optimal.}$$

That is the principle that will give your 14DOF simulation the best chance of successful circular, figure-eight, and general drifting.